

Algoritmos paralelos

En otras partes de este libro suponemos implícitamente que nuestros algoritmos se ejecutarán en una máquina que sólo puede ejecutar una instrucción a la vez. Por supuesto, cualquier máquina moderna simultanea el cálculo con las operaciones de entrada/salida tales como la espera a que se pulse una tecla, o a que se imprima un archivo. Muchas de ellas simultanean también distintas operaciones aritméticas cuando se calcula una expresión, de tal manera que las sumas, por ejemplo, se pueden efectuar en paralelo con las multiplicaciones. Sin embargo, hasta el momento no hemos considerado la posibilidad de que la máquina pudiera calcular varias docenas, o incluso varios centenares de expresiones diferentes a la vez. Si consideramos esta posibilidad, entonces podemos tener la esperanza, si somos a la vez inteligentes y afortunados, de acelerar algunos de nuestros algoritmos en la misma medida.

Las computadoras capaces de efectuar estos cálculos en paralelo no se encuentran todavía en la mesa de cualquier despacho. Sin embargo, su número crece, y el interés por los *algoritmos paralelos*, que aprovechan esta capacidad, está muy extendido. La investigación en este sentido es tan activa que no sería realista intentar mencionar todos los campos en los que se están estudiando técnicas paralelas. Consiguientemente, en este capítulo presentaremos solamente una selección introductoria de algoritmos paralelos que ilustran algunas técnicas fundamentales.

En primer lugar, describiremos con más precisión la máquina en que estamos pensando cuando se diseñan estos algoritmos. A continuación, ilustraremos una o dos técnicas básicas, y discutiremos lo que queremos decir al hablar de un algoritmo paralelo eficiente. Por último, daremos un pequeño número de ejemplos pertenecientes a los campos de teoría de grafos, de evaluación de expresiones y de ordenación.

11.1 UN MODELO PARA LA COMPUTACIÓN PARALELA

El modelo básico para la computación ordinaria, secuencial, en lo que a veces se llama una *máquina de von Neumann*, está aceptado tan ampliamente que no nos ha parecido necesario definirlo con más precisión en este libro. Todo el

mundo admite que estas máquinas ejecutan una instrucción cada vez, aplicándola a un dato cada vez, y siguiendo un programa que está almacenado en la memoria de la máquina. El hecho de que, en realidad, todas las máquinas modernas poseen un cierto grado de paralelismo, que las capacita por ejemplo para efectuar un número limitado de operaciones aritméticas en paralelo, tomar la instrucción siguiente mientras se está ejecutando la última, o simultanear la entrada y la salida con el cómputo, es esencialmente irrelevante para el modelo conceptual. De manera similar, la mayoría de los lenguajes de programación, como el lenguaje informal que empleamos en este libro, suponen que la computadora ejecuta una instrucción cada vez, y que se la aplica a una sola variable.

En el campo de la computación en paralelo, las máquinas operativas son relativamente pocas, y sus arquitecturas variadas, y no existe un consenso en lo tocante a cuál de los modelos teóricos de computación es el mejor de todos. Por ejemplo, no es evidente lo que podría implicar en la práctica seleccionar un modelo que permita a varios procesadores asignar nuevos valores a una misma variable en paralelo. Las restricciones prácticas evitarán, probablemente, que estas asignaciones en paralelo se efectúen realmente al mismo tiempo. ¿Debemos entonces suponer que ahora la variable tiene el último valor que se le haya asignado? En tal caso ¿sabemos cuál ha sido esa última asignación? O si son *realmente* posibles las asignaciones simultáneas, ¿cuál es el resultado de asignar simultáneamente varios valores a la misma variable? Estas consideraciones llevan a algunas personas a preferir un modelo que prohíba las asignaciones simultáneas múltiples a la misma variable.

Una vez más, supongamos que tenemos un vector que va a ser modificada por un cierto número de procesadores paralelos, con un procesador para cada elemento del vector. Si deseamos modelar una instrucción tal como «detenerse cuando sean nulos todos los elementos del vector», ¿cuánto tiempo debemos suponer que es necesario para implementar esta instrucción? Ciertamente, cada procesador puede ver inmediatamente que su propio elemento es cero, pero ¿cuál debemos de suponer que es el mecanismo necesario para comprobar *todos* los elementos? ¿Tienen los procesadores que intercambiar mensajes entre sí, hay una capa adicional de procesadores para el control global, o cómo se llega exactamente a una decisión?

Por estas razones, y por otras similares, no hay una respuesta clara para la pregunta acerca de qué modelo de computación en paralelo es el mejor en general. En este libro utilizamos un modelo popular llamado *máquina paralela de acceso aleatorio*, o *p-ram*. Este modelo es, ciertamente, el de uso más natural, y resulta fácil de entender, pero no es excesivamente similar a las máquinas existentes. Quede advertido el lector, por tanto, de que puede no resultar sencillo adaptar los algoritmos de este capítulo para su utilización en una máquina real. La Sección 11.11 contiene indicaciones de computadoras paralelas más realistas.

En el modelo *p-ram*, se supone que hay procesadores secuenciales que comparten una memoria global. Cada procesador posee el conjunto habitual de

instrucciones aritméticas y lógicas, instrucciones que puede ejecutar en paralelo con lo que esté sucediendo en los demás procesadores. Sin embargo, suponemos que los procesadores no se ponen a hacer cosas distintas, sino que todos ellos ejecutan un mismo programa que se les suministra desde algún punto central de control, aunque posiblemente con distintos datos. Por razones evidentes, este modelo se llama también de *instrucción única y múltiples flujos de datos*. Por el momento, no definiremos con más precisión lo que queremos decir con esto, y nos basaremos en ejemplos para clarificar una idea que en lo básico es sencilla.

Todo procesador tiene acceso a toda la memoria global. En cada paso, puede o bien leer o bien escribir en una sola posición de memoria. A nuestros efectos, supondremos además que aun cuando de una posición de memoria puedan leer cuantos procesadores lo deseen en un mismo instante, en esa posición no pueden escribir dos procesadores simultáneamente, ni tampoco es posible que un procesador lea una posición de memoria en la que se está escribiendo en el mismo instante. Así evitamos tener que decidir lo que sucede si dos o más procesadores intentan escribir valores diferentes en la misma posición, o lo que pasa si cambia un valor mientras un procesador lo está leyendo. Los modelos definidos de esta manera se llaman modelos de *lectura concurrente y escritura exclusiva*, o CREW¹. Otros posibles modelos, que no consideraremos por el momento, son el EREW (lectura exclusiva, escritura exclusiva) y el CRCW (lectura concurrente, escritura concurrente), que se definen de la forma natural. Nadie parece haber encontrado aplicaciones para un modelo de lectura exclusiva y escritura concurrente.

Cuando se analicen algoritmos paralelos en las secciones siguientes, haremos una suposición crucial: que el acceso a memoria en nuestro hipotético *p-ram* CREW, sea para leer o para escribir, se puede hacer en un tiempo constante, independientemente del número de procesadores utilizados. Esta suposición no es cierta en la práctica. Dado que no es factible proporcionar accesos directos en *hardware* desde todos los procesadores hasta todas las posiciones de memoria, el tiempo medio necesario para efectuar un acceso a memoria en un sistema real crece a medida que asciende el número de procesadores; además, algunos esquemas de acceso a memoria son más rápidos que otros. De hecho, ni siquiera es cierto que un único procesador pueda acceder a todas las direcciones de un espacio arbitrariamente grande en un tiempo constante. Por sencillez, sin embargo, en este libro ignoraremos esta complicación.

Por sencillez, también, ignoraremos la mayoría de los problemas planteados por el control global de la máquina paralela. Para describir nuestros algoritmos paralelos, utilizamos instrucciones de la forma general

para $x \in S$ hacer en paralelo $\text{instrucción}(x)$

Esto quiere decir que asignamos un procesador a cada elemento x del conjunto S , y que entonces se ejecutan las instrucciones de $\text{instrucción}(x)$ para cada uno de

estos elementos en paralelo, utilizando x como datos, y calculando en el procesador asignado. Suponemos que los procesadores se numeran en secuencia, y que los elementos del conjunto S también se pueden numerar de forma sencilla, para que la asignación de un procesador concreto a x se pueda hacer en un tiempo constante. Se utilizarán variaciones de esta forma de instrucción sin más explicaciones si está claro su significado. Por ejemplo:

para $1 \leq i \leq 10$ hacer en paralelo $\text{instrucción}(i)$

significa que hay que ejecutar $\text{instrucción}(i)$ en paralelo para $i = 1, 2, \dots, 10$.

Hay que tener cuidado para que la *instrucción* que es preciso ejecutar respete nuestro requisito (bastante impreciso por el momento) de que todos los procesadores «ejecutan la misma instrucción al mismo tiempo». Por ejemplo, no permitimos que *instrucción* sea una llamada a una función, porque esto podría dar lugar a acciones bastante diferentes por parte de cada procesador. Sin embargo, las instrucciones de asignación, los accesos a matrices y demás son aceptables en general. También admitiremos instrucciones condicionales, pero no bucles cuya longitud dependa de los datos. De esta manera, nuestro requisito va estando más claro: al decir «la misma sentencia» queremos decir más o menos «la misma instrucción de máquina», y no «la misma sentencia en algún lenguaje de alto nivel». Aceptaremos, sin embargo, que en el caso de una instrucción condicional, por ejemplo, algunos de los procesadores puedan ejecutar la instrucción aunque otros la pasen por alto.

Si un cálculo necesita utilizar p procesadores, podemos suponer además que se necesita un tiempo que está en $\Theta(\log p)$ previo al comienzo de la computación, para enviarles las instrucciones necesarias y hacer que empiecen a trabajar. Esto se consigue fácilmente si inicialmente está activo un procesador, y después en cada paso temporal cada procesador activo activa a otro más, de tal modo que el número de procesadores activos se doble a cada paso. Sin embargo, en los párrafos siguientes no tomamos en cuenta este tiempo de inicialización en forma explícita.

11.2 TÉCNICAS BÁSICAS

11.2.1 Cómputo con un árbol binario completo

La mejor manera de explicar esta sencilla técnica es a través de un ejemplo. Supongamos que deseamos calcular la suma de n enteros. Para evitar complicaciones, supongamos también que n es una potencia de 2; de no ser así, basta con añadir los elementos nulos, mudos, que sean necesarios. Estos n elementos se sitúan en las hojas de un árbol binario completo, según se ilustra en la figura 11.1. Ahora, en el primer paso, se calculan en paralelo las sumas de los elementos de nivel 1; en el segundo paso, se calculan en paralelo las sumas de los elementos del nivel 2; y así sucesivamente hasta que en el paso $(\lg n)$ -ésimo el valor que se obtiene en la raíz es la solución de nuestro problema.

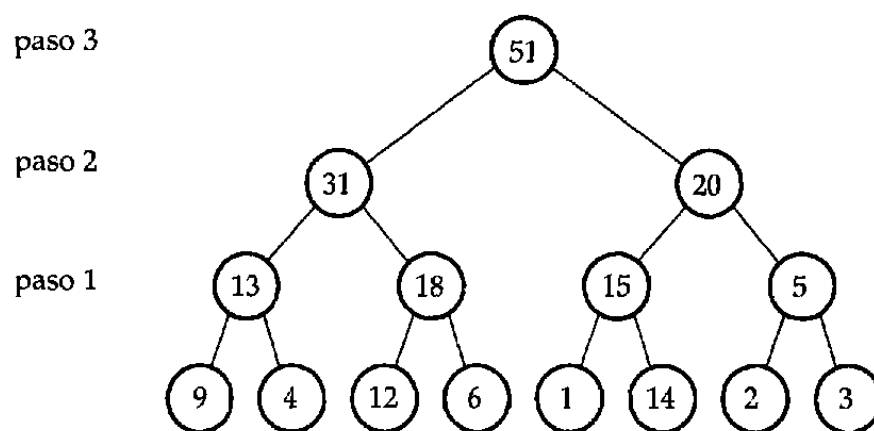


Figura 11.1. Cálculo con un árbol binario completo

Más formalmente, sea $n = 2^k$, y sea T un vector con índices en el rango 1 a $2n-1$. Este vector se puede emplear para almacenar un árbol binario completo con $2n-1$ nodos, con la raíz en $T[1]$ y con los hijos del nodo $T[i]$ en los nodos $T[2i]$ y $T[2i+1]$, exactamente igual que un montículo. Supongamos que los n elementos que hay que sumar están situados inicialmente en las hojas del árbol, esto es, en los nodos que van desde $T[n]$ hasta $T[2n-1]$. Ahora, el algoritmo para calcular la suma de estos n elementos es como sigue:

función *sumapar*(T, n)

{Calcula la suma $T[n] + \dots + T[2n-1]$ }

para $i \leftarrow \lg n - 1$ **bajando** -1 **hasta** 0 **hacer**

para $2^i \leq j \leq 2^{i+1} - 1$ **hacer en paralelo**

$T[j] \leftarrow T[2j] + T[2j+1]$

devolver $T[1]$

En este caso la única sincronización que se requiere es que todos los cálculos paralelos para un valor particular de i deben haber finalizado antes de que comiencen los cálculos correspondientes al próximo valor de i . Durante una pasada por el bucle i , cada procesador implicado efectúa dos accesos a memoria para leer sus operandos, una adición y un acceso final a memoria para almacenar el resultado. Dado que suponemos que un acceso a memoria requiere un tiempo constante, el trabajo total necesario por parte de cada procesador también requiere un tiempo constante. Por último, dado que los procesadores funcionan en paralelo, todo el trabajo necesario para una pasada por el bucle se puede ejecutar en un tiempo constante. El algoritmo efectúa $\lg n$ pasadas por el bucle, así que el tiempo total requerido está en $\Theta(\lg n)$. Es evidente que el máximo número de procesadores que tienen que funcionar simultáneamente es $n/2$.

Está claro que se puede aplicar la misma técnica a problemas tales como buscar el producto, el máximo o el mínimo de n elementos, o para decidir si son todos nulos.

11.2.2 Duplicación de punteros

El algoritmo de duplicación de punteros, en su forma más sencilla, puede aplicarse a listas de elementos. Supongamos que se dispone de una lista L de n elementos, cada uno de los cuales contiene un puntero de su sucesor. Supongamos que el sucesor del elemento i es $s[i]$. Si el elemento k es el último elemento de la lista, entonces $s[k]$ es el puntero especial **nil**. Como primer ejemplo, queremos calcular para cada elemento i la distancia $d[i]$ desde ese elemento hasta el final de la lista. Suponemos que están disponibles tantos procesadores como elementos haya en L , así que podemos asociar un procesador distinto a cada elemento de la lista. Ahora el algoritmo es como sigue:

```

procedimiento distpar( $L$ )
  { Iniciación }
  para cada elemento  $i \in L$  hacer en paralelo
    si  $s[i] = \text{nil}$  entonces  $d[i] \leftarrow 0$ 
    sino  $d[i] \leftarrow 1$ 
  { Bucle principal }
  repetir  $\lceil \lg n \rceil$  veces
    para cada elemento  $i \in L$  hacer en paralelo
      si  $s[i] \neq \text{nil}$  entonces
         $d[i] \leftarrow d[i] + d[s[i]]$ 
         $s[i] \leftarrow s[s[i]]$ 

```

La figura 11.2 ilustra el progreso de este algoritmo para una lista de 7 elementos. Tal como está escrito, los campos de punteros de la lista original se modifican, y se destruye la estructura de la lista. Si no se desea esto, se copian los punteros en la fase de iniciación del algoritmo, y después se trabaja con las copias.

Aquí la sincronización es más sutil que en el ejemplo anterior. No hay problema con los intentos simultáneos de escribir en una misma posición, puesto que cada procesador asigna solamente valores a d y a s en su propio elemento. En el modelo que hemos adoptado, no es preciso preocuparse por los intentos simultáneos de leer en la misma posición. Sin embargo, sí hay que preocuparse por los valores que cambian antes de que hayamos tenido tiempo de leerlos. Cuando varios procesadores están ejecutando la instrucción $d[i] \leftarrow d[i] + d[s[i]]$ en paralelo, por ejemplo, la sincronización debe ser tan fuerte que nos permita asegurar que el procesador asignado al elemento i va a leer el valor necesario de $d[s[i]]$ antes de que el procesador asignado al elemento $s[i]$ cambie su valor de d . La forma más segura de asegurar esto para todo i es insistir en que *todas* las lecturas necesarias para evaluar las expresiones del lado derecho se ejecuten antes de haber escrito ningún valor nuevo en las variables del lado izquierdo. De manera similar, en la instrucción $s[i] \leftarrow s[s[i]]$, las lecturas necesarias para evaluar las expresiones del lado derecho deben de producirse todas ellas antes de que se haya escrito ningún valor

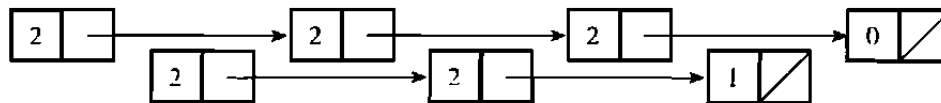
nuevo en las variables del lado izquierdo. El requisito enunciado anteriormente, de que todos los procesadores estén trabajando en la misma instrucción al mismo tiempo, tiene que interpretarse ahora de manera más estricta, con sincronización a nivel de las instrucciones de máquina.

Clave: $\begin{array}{|c|c|} \hline d[i] & s[i] \\ \hline \end{array}$ $\begin{array}{|c|c|} \hline d[i] & \diagup \\ \hline \end{array}$ representa el puntero especial **nil**

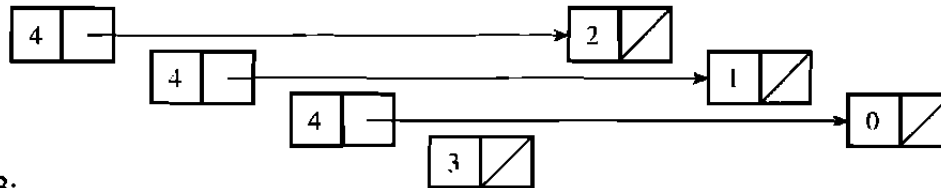
Iniciación:



Bucle 1:



Bucle 2:



Bucle 3:

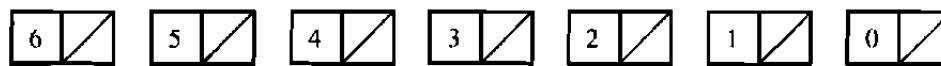


Figura 11.2. Duplicación de punteros

Cuando se detiene el algoritmo, $s[i] = \text{nil}$ para todo elemento i de L . Para ver esto, observe que los punteros se “duplican” en cada ejecución de la instrucción $s[i] \leftarrow s[s[i]]$. Más exactamente, $s[i]$ apunta originalmente al elemento que sigue a i dentro de L ; después de una ejecución de esta instrucción, $s[i]$ apunta al elemento que originalmente estaba a dos lugares de distancia de i ; después de dos ejecuciones de la instrucción, apunta al elemento que originalmente estuviera a cuatro lugares de i , y así sucesivamente. Cuando un puntero se sale de la lista le damos el valor especial **nil**. Dado que en L hay n elementos, basta duplicar la longitud de los punteros $\lceil \lg n \rceil$ veces para asegurarnos de que todos se «salen» por un extremo (para el caso en que n sea desconocido, véase el problema 11.21).

Para ver que los valores calculados de $d[i]$ son correctos, observe que al principio de cada iteración, si sumamos los valores de d para todos los elementos de la sub-lista encabezada por el elemento i (empleando, por supuesto, los valores actuales de s), obtenemos la distancia desde i hasta el final de la lista original L . Ahora, en cada iteración el puntero $s[i]$ se modifica de tal manera que se omita el sucesor inmediato de i en esta sub-lista. Sin embargo, el valor de d para este sucesor inmediato se suma a $d[i]$, así que sigue siendo cierta la misma condición al principio de la iteración siguiente.

Manteniendo nuestra suposición de que todos los accesos a memoria requieren un tiempo constante, vemos inmediatamente que el trabajo requerido por parte de cada procesador en una iteración del bucle **repetir** es constante, así que el tiempo de ejecución para el algoritmo completo está en $\Theta(\log n)$. Hay un procesador por cada elemento de la lista, así que el número total de procesadores requeridos es n .

Si la estructura de datos original no es una lista, sino una estructura de partición (véase la Sección 5.9), se puede aplicar un algoritmo similar. Supongamos que se representa cada conjunto mediante un árbol, tal como en la figura 5.21, con punteros que van desde cada elemento hasta su predecesor, salvo que las raíces se apuntan a sí mismas. Ahora se aplica un algoritmo similar a *distpar* salvo que omite toda mención a las distancias d ; este algoritmo «aplanar» los árboles de tal manera que todos los elementos señalen directamente a la raíz adecuada. Se puede ver esto como un ejemplo extremo de compresión de rutas. El algoritmo es el siguiente.

procedimiento *aplanar*(d)

{ D es una estructura de subconjunto disjunto con n elementos}

repetir $\lceil \lg n \rceil$ veces

para todo elemento $i \in D$ **hacer en paralelo**

$s[i] \leftarrow s[s[i]]$

Se utilizan variantes del método de duplicación de punteros para muchos otros cálculos sencillos. Supongamos por ejemplo que todo elemento i de la lista L tiene un valor $v[i]$, y sea $\circ$ un operador binario asociativo que toma como operandos estos valores. Considere la variante siguiente del procedimiento *distpar*.

procedimiento *operpar*(L)

{Inicialización}

para todo elemento $i \in L$ **hacer en paralelo**

$d[i] \leftarrow v[i]$

{Bucle principal }

repetir $\lceil \lg n \rceil$ veces

para todo elemento $i \in L$ **hacer en paralelo**

si $s[i] \neq \text{nil}$ **entonces**

$d[i] \leftarrow d[i] \circ d[s[i]]$

$s[i] \leftarrow s[s[i]]$

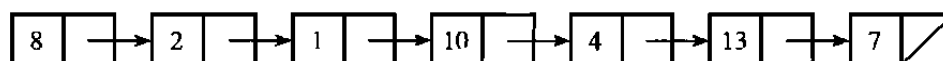
Sea x_i el valor de v para el i -ésimo elemento de la lista, con $1 \leq i \leq n$. (Esto *no* es lo mismo que $v[i]$). En el caso de x_i , el subíndice indica que deseamos el valor del i -ésimo elemento de la lista original; en el caso de $v[i]$, el índice es un puntero a un elemento, y no su posición dentro de la lista.) Se define $X_{i,j}$ mediante

$$X_{ij} = x_i \circ x_{i+1} \circ \cdots \circ x_j,$$

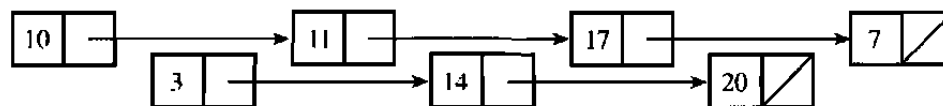
esto es, como el producto generalizado de los elementos de L desde el i -ésimo hasta el j -ésimo. Entonces no resulta difícil probar (véase el Problema 11.3) que cuando termina el algoritmo, el valor de d para el primer elemento de L es $X_{1,n}$, el valor de d para el segundo elemento de L es $X_{2,n}$, y así sucesivamente, hasta el último elemento de L cuyo valor de d es $X_{n,n} = x_n$. La figura 11.3 ilustra el funcionamiento del algoritmo para una lista de siete elementos en la cual el operador $\circ$ es la suma habitual.

Clave: $\boxed{d[i] \mid s[i]}$ $\boxed{d[i] \mid \diagup}$ representa el puntero especial **nil**

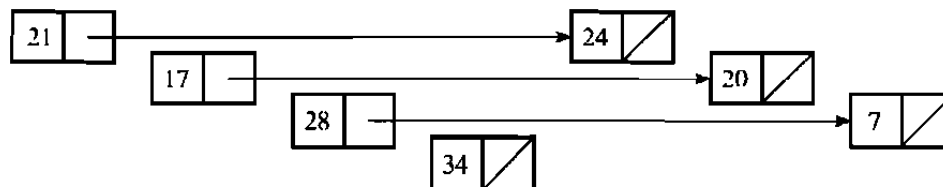
Iniciación:



Bucle 1:



Bucle 2:



Bucle 3:

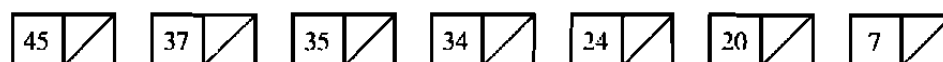


Figura 11.3. El algoritmo *operpar*

Seleccionando un operador adecuado, se pueden efectuar varios cálculos de utilidad. Tomando el operador $\circ$ como operador suma, tal como en el ejemplo de la figura 11.3, se nos da la suma de los elementos de L . Cuando se detiene el algoritmo, se proporciona la suma como valor de d para el primer elemento de L . El producto de los elementos de L , su máximo o su mínimo se pueden calcular de manera semejante. También son posibles unos esquemas más imaginativos. Por ejemplo, si cada $d[i]$ es inicializado con un puntero al propio elemento i en sí, y $\circ$ es un operador tal que al recibir dos punteros proporciona un puntero del elemento que tenga el mayor valor de v , entonces *operpar* se puede utilizar para obtener un puntero al elemento de L que tenga el mayor valor, y no solamente el valor en sí. Para un mayor desarrollo de este tema, véase el problema 11.5.

El análisis de *operpar* es exactamente igual que el de *distpar*. El tiempo de ejecución del algoritmo, siempre y cuando el operador $\circ$ se pueda tratar como elemental, está en $\Theta(\log n)$, y el número de procesadores necesarios es n .

11.3 TRABAJO Y EFICIENCIA

Parece probable que un algoritmo paralelo que tarde t segundos en ejecutarse con digamos 20 procesadores efectue más cálculos que uno que tarde el mismo tiempo empleando sólo cinco procesadores, que a su vez trabaja más que un algoritmo secuencial ordinario que también requiera t segundos. Por esta razón, definimos el *trabajo* efectuado por un algoritmo, tanto si es secuencial como si es paralelo, como el producto pt del número p de procesadores que utiliza por el tiempo de ejecución t . (Dado que para muchos algoritmos el número de procesadores necesarios varía durante el cómputo, se puede argumentar que el trabajo efectuado se mediría mejor mediante $\sum it_i$, en donde t_i es el tiempo durante el cual están activos exactamente i procesadores). Sin embargo, esto supone implícitamente que los procesadores no utilizados quedan libres para hacer alguna otra cosa. En todo caso, es demasiado complicado para lo que necesitamos en este momento). Se ve fácilmente que para un algoritmo paralelo este trabajo es el tiempo necesario para simular el algoritmo paralelo empleando un único procesador, que en cada paso del cómputo imita por turno a cada uno de los procesadores paralelos. Si tenemos dos algoritmos A y B para un mismo problema, que requieren trabajos w_a y w_b respectivamente para obtener una solución, diremos que A es *eficiente en trabajo* con respecto a B si $w_a \in O(w_b)$.

En la Sección 12.5.1 veremos que un algoritmo ordinario, secuencial, se suele considerar eficiente si su tiempo de ejecución para un problema de tamaño n está en $O(n^k)$ para alguna constante k . Por otra parte, para que se considere eficiente un algoritmo paralelo, esperamos normalmente que satisfaga dos restricciones, una tocante al número de procesadores y otra que concierne al tiempo de ejecución. Ha de cumplir:

- ◇ el número de procesadores necesarios para resolver un caso de tamaño n debe estar en $O(n^a)$ para alguna constante a , y
- ◇ el tiempo requerido para resolver un caso de tamaño n debe estar en $O(\log^b n)$ para alguna constante b .

Diremos que un algoritmo paralelo eficiente requiere un número polinómico de procesadores, y un tiempo *polilogarítmico*.

Un algoritmo paralelo se dice *óptimo* si es eficiente en trabajo respecto al mejor algoritmo secuencial posible. A veces se puede denominar *óptimo* si es tarea eficiente con respecto al mejor algoritmo secuencial *conocido*. En este caso, sin embargo, es preferible decir que el problema correspondiente tiene *aceleración óptima*. Veremos en el Capítulo 12 que hay muchos problemas para los cuales no se conoce ningún algoritmo secuencial eficiente (esto es, de tiempo polinómico). Para tales problemas, no podemos esperar hallar una solución paralela eficiente (esto es, una que utilice un número polinómico de procesadores y un tiempo polilogarítmico): véase el problema 11.6. Por otra parte, hay muchos problemas para los cua-

les se conoce un algoritmo secuencial eficiente, pero para los cuales todavía no se ha descubierto un algoritmo paralelo eficiente. Se cree, aunque no se ha demostrado, que algunos problemas que se pueden resolver mediante un algoritmo secuencial eficiente no poseen una solución paralela eficiente.

Considérese la técnica descrita en la Sección 11.2.1 como ejemplo. Vimos allí que se puede calcular la suma de n elementos almacenados en un vector empleando $n/2$ procesadores y utilizando un tiempo que está en $\Theta(\lg n)$. El trabajo que requiere el algoritmo está por tanto en $(n/2) \times \Theta(\lg n) = \Theta(n \lg n)$. Puesto que la suma de n elementos se puede obtener claramente en $\Theta(n)$ operaciones por simple adición en un procesador secuencial, el algoritmo paralelo, aunque es eficiente, no es óptimo. De manera similar, las técnicas descritas en la Sección 11.2.2 nos permiten efectuar toda una variedad de operaciones (calcular la distancia hasta el extremo de una lista de n elementos, hallar la suma o el máximo de la lista de elementos, etc.) en un tiempo que está en $\Theta(\lg n)$ empleando n procesadores. Una vez más, el trabajo necesario está en $\Theta(n \lg n)$. Puesto que estas operaciones se pueden efectuar mediante un solo procesador secuencial en un tiempo que está en $\Theta(n)$, estos algoritmos tampoco son óptimos.

Si examinamos con más detalle el algoritmo necesario para calcular la suma de n elementos empleando un árbol binario, una posible razón para que no llegue a ser óptimo es patente a primera vista. En la primera pasada por el bucle principal, se necesitan $n/2$ procesadores, y esto determina los recursos necesarios para el algoritmo, porque en la segunda pasada por el bucle sólo hay $n/4$ procesadores que realicen un trabajo útil; en la tercera sólo se necesitan $n/8$ y así sucesivamente: durante la mayor parte del tiempo, casi todos los procesadores están ociosos. Esto sugiere que quizá seamos capaces de utilizar menos procesadores sin que esto tenga un efecto catastrófico sobre el tiempo de cómputo.

Supongamos que sólo están disponibles $p < n/2$ procesadores. Una forma de proceder consiste en dividir en p grupos los números cuya suma haya que calcular; $p-1$ grupos contienen n/p números, mientras que el último contiene los $n - (p-1)\lceil n/p \rceil$ números. El último grupo, por tanto, puede contener menos de $\lceil n/p \rceil$ miembros, pero nunca puede contener más. Ahora asignamos a cada grupo uno de los procesadores disponibles, y ponemos a cada procesador a calcular la suma de su grupo. Aun cuando los procesadores trabajan en paralelo, los cálculos individuales pueden ser simples cálculos secuenciales que requerirán $\Theta(\lceil n/p \rceil)$ operaciones. Dado que los procesadores trabajan en paralelo, el tiempo total requerido para esta fase también está en $\Theta(\lceil n/p \rceil)$. El problema se reduce ahora a hallar la suma de las p sumas de grupo, y esto se puede resolver mediante la técnica del árbol equilibrado sin modificación empleando $p/2$ procesadores en un tiempo que está en $\Theta(\lg p)$. En total, el algoritmo modificado que emplea $p < n/2$ procesadores requiere un tiempo que está en $\Theta(\lceil n/p \rceil + \lg p)$.

En particular, si hacemos $p = n/\log n$ obtenemos un algoritmo que puede calcular la suma de n números en un tiempo que está en $\Theta(\lg n)$. Por tanto hemos reducido el número de procesadores necesarios por un factor $(\log n)/2$, sin modi-

ficar el orden del tiempo de ejecución. El trabajo efectuado por el algoritmo modificado está en $\Theta(p \times \log n) = \Theta(n)$. Es claro que ningún algoritmo secuencial puede hacerlo mejor que esto, así que el algoritmo modificado es un algoritmo paralelo óptimo.

En general, podemos no ser tan afortunados. Por ejemplo, dividir los elementos de una lista en grupos es más difícil que dividir los elementos de un vector. Para la primera, podemos tener que empezar por explorar toda la lista; para el segundo, suele ser suficiente un cálculo sencillo haciendo uso de los índices del vector. Sin embargo, empleando una técnica similar siempre podemos reducir el número de procesadores requeridos por un algoritmo paralelo. Supongamos que tenemos un algoritmo que funciona en un tiempo t empleando p procesadores para un problema de tamaño n , pero que sólo disponemos de $q < p$ procesadores (aquí, t , p y q son funciones de n). ¿Qué deberíamos hacer?

Tal como antes, dividimos los p procesadores en q grupos, y utilizamos uno de los q procesadores disponibles para simular cada grupo. Habrá $q-1$ grupos que contengan p/q procesadores, y al menos un grupo que no contendrá más procesadores que los otros, y quizá menos. A continuación, supongamos que el algoritmo original efectúa los pasos $1, 2, \dots$, en donde los p procesadores ejecutan independientemente cada paso, pero es preciso sincronizarlos entre pasos. En el algoritmo modificado que emplea solamente q procesadores, en el paso primero uno de ellos simula por turnos a cada procesador del primer grupo; un segundo procesador simula por turnos a cada procesador del segundo grupo, y así sucesivamente. Dado que hay p/q procesadores, o menos, que simular en cada grupo, la simulación del paso 1 empleando q procesadores requiere p/q veces más que en el paso 1 original, y lo mismo sucede para los demás pasos. De esta manera, el cálculo completo empleando q procesadores requiere p/q veces más tiempo que el cálculo completo empleando p procesadores. En símbolos, el algoritmo modificado requiere un tiempo que está en $\Theta(\lceil p/q \rceil t)$. (Recuerde que p , q y t pueden ser funciones del tamaño n del caso.) Dado que $p/q \leq \lceil p/q \rceil < 2p/q$ cuando $p > q$, hemos demostrado el teorema siguiente.

Teorema 11.3.1 (Brent) *Si existe un algoritmo paralelo que requiere un tiempo $t(n)$ para resolver un problema de tamaño n empleando $p(n)$ procesadores, entonces para cualquier $q(n) < p(n)$ existe un algoritmo modificado que puede resolver el mismo problema empleando solamente $q(n)$ procesadores en un tiempo $\Theta(p(n)t(n)/q(n))$.*

Aquí, el algoritmo original funciona ciertamente en $\Theta(p(n)t(n)/q(n) \times q(n)) = \Theta(p(n)t(n))$, así que en términos de trabajo no hemos ganado ni perdido con la modificación: el algoritmo modificado es eficiente en trabajo con respecto al algoritmo original. Por tanto, podemos reducir el número de procesadores empleados por un algoritmo sin reducir su eficiencia. En particular, si el algoritmo original es óptimo, entonces también lo es el algoritmo modificado. Por supuesto, el algoritmo

mo que utilice menos procesadores tardará normalmente más tiempo en terminar, aun cuando el trabajo efectuado sea el mismo.

11.4 DOS EJEMPLOS DE TEORÍA DE GRAFOS

11.4.1 Caminos mínimos

Ya hemos encontrado variantes de este problema en las Secciones 6.4 y 8.5. Repetiremos brevemente los detalles. Sea $G = \langle N, A \rangle$ un grafo dirigido. Los nodos de G están numerados de 1 a n , y hay una matriz L que da las longitudes de las aristas, con $L[i, i] = 0$; $L[i, j] \geq 0$ si $i \neq j$, y $L[i, j] = \infty$ si no existe la arista (i, j) . Deseamos calcular la longitud de el camino más corto entre todos los pares de nodos. Ahora daremos un algoritmo paralelo para este problema. Es interesante comparar esto con el algoritmo *Floyd* de la Sección 8.5.

El algoritmo paralelo construye una matriz D que da la longitud del camino más corto entre todos los pares de nodos. Da a D el valor inicial L , esto es, las distancias directas entre nodos, y después efectúa $\lceil \lg n \rceil$ iteraciones. Después de la iteración k , D da la longitud de los caminos mínimos que no utilizan más de 2^k aristas, o equivalentemente, que no utilizan más de $2^k - 1$ nodos intermedios.

Como las longitudes de las aristas son no negativas, los caminos mínimos que estamos buscando deben de ser simples: esto es, no pueden visitar dos veces el mismo nodo. Por tanto, utilizan como máximo $n - 1$ aristas. Consiguientemente, al cabo de $\lceil \lg n \rceil$ iteraciones, D da el resultado deseado.

En la iteración k , el algoritmo debe de comprobar para cada pareja de nodos (i, j) si existe o no una nuevo camino desde i hasta j que utilice más de 2^{k-1} aristas pero menos de 2^k , y que sea mejor que el presente camino óptimo, que no utiliza más de 2^{k-1} aristas. Cualquiera de estas caminos nuevos tiene un nodo «medio» llamado m definido de tal manera que ni la parte de el camino que va de i a m ni la que va de m a j utilicen más de 2^{k-1} aristas. Las longitudes óptimas de estas partes son por tanto los valores actuales de $D[i, m]$ y de $D[m, j]$, respectivamente. Dado que es aplicable el principio de optimalidad, para comprobar si existe un nuevo camino mejorado desde i hasta j , basta comparar la longitud del mejor camino existente con $D[i, m] + D[m, j]$ para todos los posibles valores de m . Entonces nos limitamos a seleccionar el mínimo. Véase el algoritmo.

procedimiento *caminospar*($L[1..n, 1..n]$): **matriz**[$1..n, 1..n$]

matriz $D[1..n, 1..n]$, $T[1..n, 1..n, 1..n]$

para todo i, j **hacer en paralelo** $D[i, j] \leftarrow L[i, j]$

repetir $\lceil \lg n \rceil$ **veces**

para todo i, j, m **hacer en paralelo**

$T[i, m, j] \leftarrow D[i, m] + D[m, j]$

para todo i, j **hacer en paralelo**

$D[i, j] \leftarrow \min(D[i, j], T[i, 1, j], T[i, 2, j], \dots, T[i, n, j])$

devolver D

Aquí la matriz T contiene las longitudes de los caminos para evitar conflictos entre lecturas y escrituras en la última instrucción **para**. No hay conflicto entre la lectura del viejo valor de $D[i, j]$ y la escritura de su nuevo valor en esta instrucción, puesto que esto es efectuado por el mismo procesador. Las variables i, j y m varían entre 1 y n .

El análisis de este algoritmo es sencillo. La primera instrucción **para** se puede ejecutar en un tiempo constante empleando n^2 procesadores. Dentro de la instrucción **repetir**, la primera instrucción **para** se puede ejecutar en tiempo constante empleando n^3 procesadores. Se puede calcular el mínimo de entre $n+1$ elementos en un tiempo que está en $\Theta(\log n)$ empleando $\Theta(n / \log n)$ procesadores, según se describe en la Sección 11.2.1. Hay que calcular n^2 de estos mínimos en paralelo, así que una iteración de la segunda instrucción **para** se puede ejecutar en un tiempo que está en $\Theta(\log n)$ empleando $\Theta(n^3 / \log n)$ procesadores. Por último, la instrucción **repetir** se ejecuta $\lceil \lg n \rceil$ veces, así que el algoritmo completo se puede ejecutar en un tiempo que está en $\Theta(\log^2 n)$ empleando $\Theta(n^3)$ procesadores.

Es sencillo demostrar que el número de procesadores se puede reducir a $\Theta(n^3 / \log n)$ aun manteniendo el orden del tiempo: véase el Problema 11.8. A pesar de ello, el algoritmo sigue sin ser óptimo

11.4.2 Componentes conexos

Sea $G = \langle N, A \rangle$ un grafo no dirigido. Como de costumbre, suponemos que los nodos de G están numerados de 1 a n . Sea un vector L tal que $L[i, j] = \text{verdadero}$ si existe el borde (i, j) , y $L[i, j] = \text{falso}$ en caso contrario. Dado que el grafo es no dirigido, $L[i, j] = L[j, i]$ para todo par de nodos (i, j) . Deseamos hallar los componentes conexos del grafo G . El problema 9.16 pide al lector hallar un algoritmo secuencial para este problema. Aquí describimos un algoritmo paralelo.

En resumidas cuentas, el algoritmo actúa formando conjuntos disjuntos (véase la Sección 5.9) de nodos de los que sabemos que están conectados, fusionando estos conjuntos con otros más grandes, y así sucesivamente, hasta que finalmente los nodos de cada componente conexo del grafo se encuentran en un único conjunto. Asociamos con cada nodo de G una entrada del vector *conjunto*[1.. n]. Si *conjunto*[i]= i , entonces i es a la vez el rótulo de un conjunto y la raíz del árbol correspondiente; si *conjunto*[i] = $j \neq i$, entonces el nodo i está en el conjunto cuyo rótulo es j , pero no la raíz del árbol. Obsérvese que en esta aplicación:

- (a) los árboles siempre están «aplanados»; esto es, cada nodo (incluyendo la raíz en sí) apunta directamente a la raíz; y
- (b) la etiqueta del conjunto, esto es, el nodo que se encuentra en la raíz del árbol, siempre es el nodo de número más bajo en todo el conjunto.

Comenzaremos por describir tan solo una iteración del algoritmo paralelo que fusiona conjuntos disjuntos. Para ilustrar el funcionamiento del algoritmo, supongamos por ejemplo que tenemos un grafo con 19 nodos, y que de una forma u otra hemos alcanzado la situación que se muestra en la figura 11.4. Aquí los nodos 1, 2

y 3 están en el conjunto marcado como 1, los nodos 4 y 5 están en el conjunto marcado como 4 y así sucesivamente. En términos de nuestra representación, esto significa que $\text{conjunto}[1] = \text{conjunto}[2] = \text{conjunto}[3] = 1$, $\text{conjunto}[4] = \text{conjunto}[5] = 4$, y así sucesivamente. Se sabe que los nodos de cualquier conjunto dado están conectados, pero también hay algunas conexiones que todavía no se han tenido en cuenta. Estas se indican mediante líneas punteadas en la figura 11.4. De esta forma, el nodo 3, por ejemplo, está conectado con el nodo 7, y el nodo 4 está conectado con el nodo 8; en otras palabras, $L[3, 7] = \text{verdadero}$, $L[4, 8] = \text{verdadero}$ y así sucesivamente, mientras que por ejemplo $L[3, 4] = \text{falso}$ porque los nodos 3 y 4 no están conectados. Se omiten las conexiones entre nodos del mismo conjunto, por carecer ya de interés.

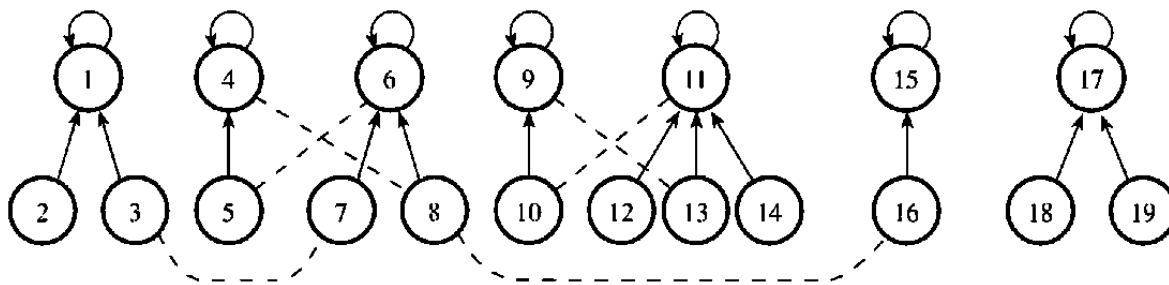


Figura 11.4. Fusión de conjuntos: situación inicial

En la descripción del algoritmo, necesitamos una matriz $S[1..n, 1..n]$, y dos vectores $T[1..n]$ y $\text{viejo}T[1..n]$. Ahora se puede especificar en la forma siguiente el primer paso del algoritmo paralelo de fusión:

{Paso 1}

para todo i, j hacer en paralelo

si $L[i, j]$ y $\text{conjunto}[i] \neq \text{conjunto}[j]$ entonces $S[i, j] \leftarrow \text{conjunto}[j]$
sino $S[i, j] \leftarrow \infty$

para todo i hacer en paralelo $T[i] \leftarrow \min(S[i, 1], S[i, 2], \dots, S[i, n])$

para todo i hacer en paralelo

si $T[i] = \infty$ entonces $T[i] \leftarrow \text{conjunto}[i]$

Tanto aquí como en el resto de esta sección, las variables i y j varían entre 1 y n .

El efecto de este paso es que para cada nodo i , ahora $T[i]$ apunta a la raíz de un conjunto. Si el nodo i está conectado a nodos de otros conjuntos además del suyo, entonces $T[i]$ apunta a la raíz de uno de estos otros conjuntos: de hecho, al que tenga el número más bajo. Si el nodo i no tiene conexiones fuera de su propio conjunto, entonces $T[i] = \text{conjunto}[i]$.

Aplicando el paso 1 a la situación de la figura 11.4 se obtiene la situación de la figura 11.5, en donde ahora se han omitido las conexiones entre nodos, y las flechas muestran los valores de T obtenidos. Todas las flechas apuntan a nodos raíz. De esta forma, por ejemplo, el nodo 1, que no tiene conexiones fuera de su propio

conjunto, tiene $T[1] = \text{conjunto}[1] = 1$. El nodo 3, que está conectado con el nodo 8, el cual no está en su propio conjunto, apunta a la raíz de este otro conjunto, a saber, el nodo 6. El nodo 8 proporciona un caso algo más complicado. Está conectado tanto al nodo 4 como al 16, ninguno de los cuales está en el mismo conjunto que el nodo 8. Entonces $T[8]$ debe apuntar o bien a la raíz del conjunto que contiene el nodo 4, o bien a la raíz del conjunto que contiene el nodo 16, esto es, o bien al nodo 4 o bien al nodo 15. Dado que el algoritmo selecciona el número más bajo, obtenemos $T[8] = 4$.

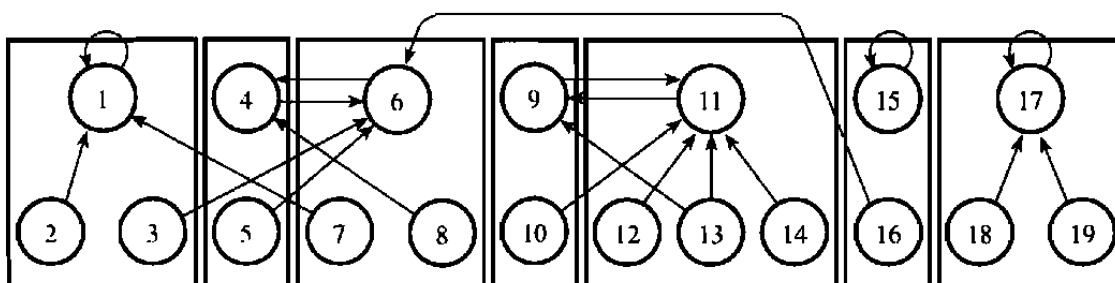


Figura 11.5. Fusión de conjuntos: después del paso 1

El segundo paso del algoritmo de fusión es como sigue.

{ Paso 2}

para todo i, j hacer en paralelo

si $\text{conjunto}[j] = i$ **y** $T[j] \neq i$ **entonces** $S[i, j] \leftarrow T[j]$

sino $S[i, j] \leftarrow \infty$

para todo i hacer en paralelo $T[i] \leftarrow \min(S[i, 1], S[i, 2], \dots, S[i, n])$

para todo i hacer en paralelo

si $T[i] = \infty$ **entonces** $T[i] = \text{conjunto}[i]$

Si el nodo i no es la etiqueta de un conjunto, no hay un nodo j que tenga $\text{conjunto}[j] = i$, así que este paso se limita a hacer $T[i] \leftarrow \text{conjunto}[i]$. Si por otra parte i es un rótulo, entonces el algoritmo examina todos los valores de $T[j]$ para los cuales j sea un nodo del conjunto i , y para los cuales $T[j] \neq i$, esto es, $T[j]$ apunta a otro conjunto diferente. Entonces selecciona el más pequeños de entre ellos. Si no existen tales valores, $T[i] \leftarrow \text{conjunto}[i]$, esto es, el rótulo de nodo i se apunta a sí mismo. Esto sólo sucede si ninguno de los nodos del conjunto i está conectado a un nodo de otro conjunto diferente.

Una vez que se aplica el paso 2 a la situación de la figura 11.5, se obtiene la situación ilustrada en la figura 11.6. Las flechas muestran ahora los nuevos valores de T . Todo nodo que no sea un rótulo apunta a la raíz de su propio conjunto, y las flechas entre conjuntos unen solamente nodos raíz.

Considere este grafo dirigido, que llamaremos H : sus nodos son los nodos de G , pero sus aristas están especificadas por los punteros T . Se ha redibuiado en la figura 11.7 para hacer más clara su estructura. Siguiendo el algoritmo, vemos que si uno de los conjuntos iniciales carece de conexión con todos los demás (esto es,

si incluye a todos los nodos de algún componente conexo del grafo G), entonces al cabo de los pasos 1 y 2, los punteros T se limitan a reproducir la estructura inicial del conjunto. Este es el caso del conjunto 17 del ejemplo. Los otros nodos de H forman uno o más componentes conexos, cada uno de los cuales se parece a una pareja de árboles cuyas raíces están unidas en un ciclo. En el ejemplo, un par de árboles tiene como raíces los nodos 1 y 6, y el otro tiene los nodos 9 y 11.

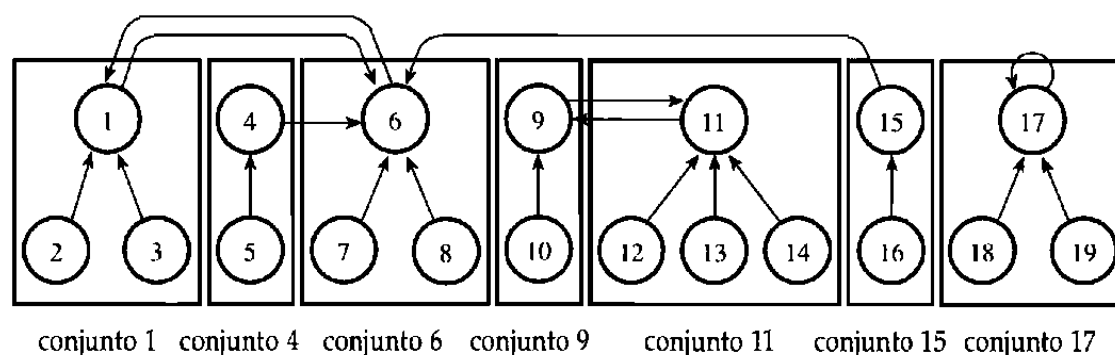


Figura 11.6. Fusión de conjuntos: después del paso 2

Para ver por qué es así, considere un componente de H formado por la fusión de dos o más de los conjuntos originales. Supongamos que a es el rótulo del conjunto de número más bajo de entre los implicados. Dado que la etiqueta del conjunto a es el nodo de número más bajo dentro del conjunto, esto significa que a es de hecho el nodo de número más bajo dentro del componente dado de H . Ahora bien, $T[a] = b$, donde b es la etiqueta de un conjunto distinto de a , puesto que en el paso 2 del algoritmo seleccionamos punteros entre conjuntos distintos siempre que es posible. Además, $T[b] = a$, puesto que si el conjunto a está conectado con el conjunto b , entonces el conjunto b está conectado con el conjunto a (porque el grafo G es no dirigido), y $T[b]$ se ha seleccionado en el paso 2 de manera que sea el menor posible. Entonces $T[a] = b$ y $T[b] = a$, y estos dos nodos forman un ciclo. Los nodos restantes de este componente de H deben estar unidos o bien al nodo a o bien al b mediante una cadena de uno o más punteros T , así que forman dos árboles, uno con a como raíz y otro con b como raíz.

El tercer y último paso del algoritmo utiliza la técnica de duplicación de punteros para aplanar estos árboles dobles, de forma bastante parecida a la Sección 11.2.2. La única sutileza es que si «doblamos» el puntero de un nodo con suficiente frecuencia, entonces estamos seguros de que apuntará a una de las dos raíces, pero no podemos estar seguros de cuál. En el ejemplo, si «doblamos» el puntero del nodo 5, apuntará al nodo 6; todas las duplicaciones subsiguientes dejan esto sin cambios. Si doblamos el puntero del nodo 4, apuntará al nodo 1; una vez más, las duplicaciones subsiguientes no alteran esto. Sin embargo, si las dos raíces son los nodos a y b , hemos visto que antes de toda duplicación de punteros es $T[a] = b$ y $T[b] = a$. Supongamos que guardamos los valores originales de T en el vector *viejoT*. Al cabo de un número suficiente de duplicaciones el puntero del nodo i apunta a una de las dos raíces, y ahora podemos utilizar los valores de *viejoT* para ha-

llar la otra. Comparando, podemos seleccionar la raíz que tenga el número menor. Tal como se indicaba más arriba, se trata del nodo de número más bajo de este componente de H .

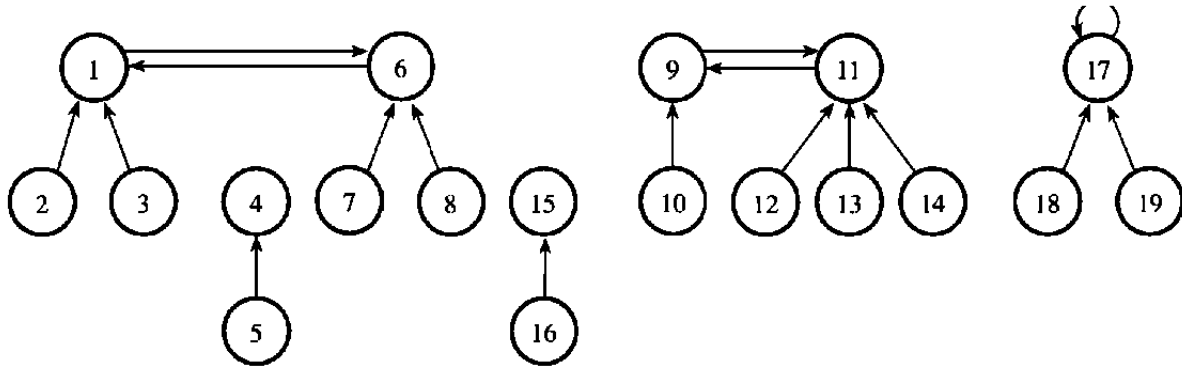


Figura 11.7. Fusión de conjuntos: el grafo H

En un componente de H que tenga una sola raíz, digamos a , al principio es $T[a] = a$, así que la técnica anterior, aun siendo innecesaria, es inofensiva. Dado que todo el grafo G contiene n nodos, ningún componente de H puede componer más de n . Tanto para un árbol sencillo como para uno doble, por tanto, bastan $\lceil \lg n \rceil$ duplicaciones para asegurar que todos los nodos apuntan a una raíz.

Éste es el tercer paso del algoritmo:

```
{ Paso 3}
para todo  $i$  hacer en paralelo  $\text{viejo}T[i] \leftarrow T[i]$ 
repetir  $\lceil \lg n \rceil$  veces
  para todo  $i$  hacer en paralelo  $T[i] \leftarrow T[T[i]]$ 
  para todo  $i$  hacer en paralelo  $\text{conjunto}[i] \leftarrow \min(T[i], \text{viejo}T[T[i]])$ 
```

Supongamos que algún componente conexo de G estuviera representado inicialmente por más de un conjunto disjunto. Tras la ejecución de los pasos 1, 2, y 3 algunos de ellos se habrán fusionado en otros más grandes. De hecho, todo conjunto que represente solamente una parte de un componente conexo de G debe de estar enlazado al menos con otro conjunto distinto que represente una parte diferente del mismo componente. Por tanto tales conjuntos deben fusionarse de dos en dos en el caso peor. En otras palabras, en el caso peor el número de conjuntos disjuntos que representan el mismo componente conexo de G se divide por dos mediante la aplicación de los pasos 1, 2 y 3.

El algoritmo paralelo completo para hallar los componentes conexos de un grafo no dirigido G con n nodos se puede especificar por tanto en la forma siguiente. En primer lugar, cada nodo de G se pone en un conjunto disjunto diferente. A continuación se iteran los pasos 1, 2 y 3 durante $\lceil \lg n \rceil$ veces. Dado que el número de conjuntos disjuntos que representan a un mismo componente conexo de G se divide al menos por dos en cada iteración, es seguro que esto efectuará todas las posibles fusiones, aun cuando G tuviera un único componente conexo. Cuando se

detenga el algoritmo, los conjuntos disjuntos restantes representarán distintos componentes conexos de G . El algoritmo es el siguiente:

procedimiento *compcons*($L[1..n, 1..n]$): **matriz** $[1..n]$
 { L es la matriz booleana de adyacencia de un grafo G
 no dirigido. El algoritmo proporciona un vector que
 representa una estructura de conjuntos disjuntos, en
 donde cada conjunto disjunto corresponde a un
 componente conexo de G }
 matriz *conjunto*[$1..n$], $S[1..n, 1..n]$, $T[1..n]$, *viejo* $T[1..n]$
 para todo i **hacer en paralelo** *conjunto*[i] $\leftarrow i$
 repetir $\lceil \lg n \rceil$ **veces**
 Paso 1
 Paso 2
 Paso 3
 devolver *conjunto*

Análisis del algoritmo

El análisis del algoritmo es sencillo. En primer lugar, se iteran los tres pasos $\lceil \lg n \rceil$ veces, y dado que cada ejecución del paso 3 da lugar a que se ejecute $\lceil \lg n \rceil$ veces la instrucción de duplicación de punteros, el tiempo de ejecución del algoritmo está en $\Omega(\log^2 n)$, independientemente del número de procesadores que estén disponibles.

Ahora se puede ejecutar la primera instrucción **para** del paso 1 en un tiempo constante, con n^2 procesadores. En la segunda instrucción **para** de este paso, el mínimo de n valores de un vector se puede hallar en un tiempo que está en $\Theta(\log n)$ empleando $\Theta(n^2)$ procesadores. La tercera instrucción **para** se puede ejecutar en tiempo constante empleando n procesadores. En consecuencia, el total del paso 1 se puede ejecutar en un tiempo que está en $\Theta(\log n)$ empleando $\Theta(n^2)$ procesadores. Es fácil comprobar que no hay conflictos de escritura durante este paso. El análisis del paso 2 es exactamente paralelo al del caso 1.

En lo tocante al caso 3, las instrucciones primera y última se pueden ejecutar en tiempo constante con n procesadores, mientras que la instrucción **repetir**, tal como se vio, requiere un tiempo que está en $\Theta(\log n)$ y se puede ejecutar con n procesadores. Una vez más, no hay conflictos de escritura.

Reuniendo todos estos hechos, vemos que se puede ejecutar el algoritmo *compcons* en un tiempo que está en $\Theta(\log^2 n)$ empleando $\Theta(n^2)$ procesadores. El trabajo efectuado se encuentra en $\Theta(n^2 \log^2 n)$, que no es óptimo. De hecho, se puede mejorar con facilidad. Los puntos críticos del algoritmo que determinan el número de procesadores necesarios son los cálculos de n mínimos en los pasos 1 y 2. Empleando el resultado del problema 11.2, vemos que lo cierto es que se puede calcular un mínimo en un tiempo que está en $\Theta(\log n)$ empleando un

número de procesadores que está en $\Theta(n/\log n)$; entonces los n mínimos se pueden calcular en el mismo tiempo empleando $\Theta(n^2/\log n)$ procesadores. Con esta mejora, el algoritmo *compcons* se puede ejecutar en un tiempo que está en $\Theta(\log^3 n)$ empleando $\Theta(n^2/\log n)$ procesadores. Sin embargo, esto sigue sin ser óptimo.

El algoritmo puede mejorarse todavía más aprovechando el hecho consistente en que a medida que progresan los cálculos va disminuyendo el número de conjuntos disjuntos que hay que manejar, y se necesitan menos procesadores. Sin embargo, esto da lugar a un algoritmo paralelo para el problema de los componentes conexos que sigue requiriendo un tiempo que está en $\Theta(\log^2 n)$, pero ahora se emplean solamente $\Theta(n^2/\log^2 n)$ procesadores. El trabajo efectuado por este algoritmo mejorado está en $\Theta(n^2)$. Dado que ningún algoritmo secuencial que utilice el vector de adyacencia del grafo G puede hacerlo mejor, el algoritmo paralelo mejorado es óptimo.

11.5 EVALUACIÓN DE EXPRESIONES EN PARALELO

Existe una notable cantidad de literatura acerca de este tema, que no pretendemos resumir aquí. Daremos tan solo un ejemplo de la forma de computar expresiones sencillas en paralelo. El ejemplo seleccionado es fácil de explicar; sin embargo, la técnica de resolución resulta gratamente novedosa.

Supongamos que se está buscando un algoritmo paralelo para evaluar expresiones aritméticas sencillas, cuyos operandos son constantes, y que solamente involucran a los cuatro operadores $+$, $-$, $\times$ y $/$. A lo largo de esta sección supondremos que estas cuatro operaciones aritméticas son elementales, esto es, que se pueden calcular en tiempo constante. Por sencillez, suponemos que la expresión que hay que evaluar se encuentra en la forma de un *árbol de expresión*: se trata de un árbol binario en el que cada nodo interno representa uno de los cuatro operadores disponibles, y cada hoja denota un operando. La figura 11.8 muestra uno de estos árboles, correspondiente a la expresión

$$\{(- (2), 3), \times 3\} + \{(9 \times (10 - 8))\} + 6$$

Además, suponemos que las hojas del árbol están numeradas de izquierda a derecha siguiendo el borde inferior del árbol, también según se ilustra en la figura 11.8. La n siguiendo el borde inferior que hay a la derecha de los nodos internos de esta figura se explicará más adelante. Si el árbol tiene n hojas (operandos) entonces desde luego tiene $n-1$ nodos (operadores). En el ejemplo, $n = 8$.

La forma evidente de evaluar paralelamente uno de estos árboles de expresión consiste en asignar un procesador a cada nodo interno, y utilizar un algoritmo de la forma siguiente:

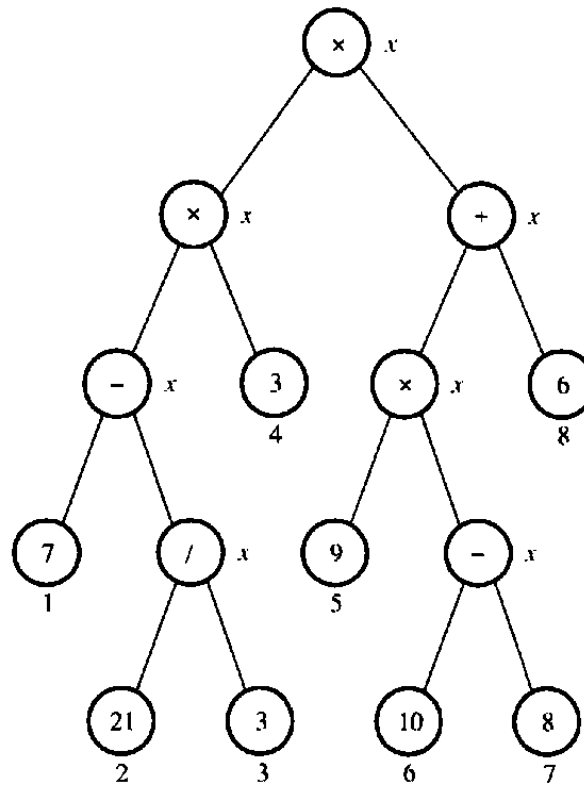


Figura 11.8. Un árbol de expresión binaria.

repetir

para cada nodo interno i **hacer en paralelo**

si los valores de los hijos de i son conocidos **entonces**

calcular el valor de i

eliminar del árbol los hijos de i

hasta que sólo quede un nodo

El número de iteraciones necesarias es igual a la altura del árbol de expresión. En el caso peor, sin embargo, un árbol binario con n hojas puede tener una altura $n-1$, y este algoritmo sencillo ¡no produce paralelismo alguno! Sólo un procesador efectúa un trabajo útil en cada iteración, y para eso el cálculo podría hacerse en una máquina secuencial; véase el problema 11.10.

Para acelerar las cosas, los procesadores deben de hacer algo útil incluso antes de que se hayan evaluado sus dos hijos. Aquí buscamos algo que pueda hacer un procesador cuando se conoce el valor de al menos uno de sus hijos. Con este objeto, asociamos una función $f(x)$ a cada nodo interno del árbol. Inicialmente, todos los nodos internos están asociados con la función x , según se ilustra en la figura 11.8. El significado de estas funciones es que cuando un procesador de algún nodo ha calculado un valor x , el valor que transmite hacia arriba por el árbol no es x sino $f(x)$. Considere por ejemplo el fragmento de árbol que se muestra en la Figura 11.9a. Aquí el procesador asignado al nodo interno A recibe un valor de su hijo izquierdo, y un valor del derecho, los multiplica para obtener una respuesta x , y después transmite el valor $f(x)$ hacia arriba, al nodo B . A su vez, el procesador asociado al nodo B recibe este valor de su hijo izquierdo A y el valor 9 de su hijo derecho C (que es una hoja, y corresponde

a un operando constante), los suma para obtener una respuesta x y transmite entonces el valor $g(x)$ por el árbol hasta su predecesor. Y así sucesivamente.

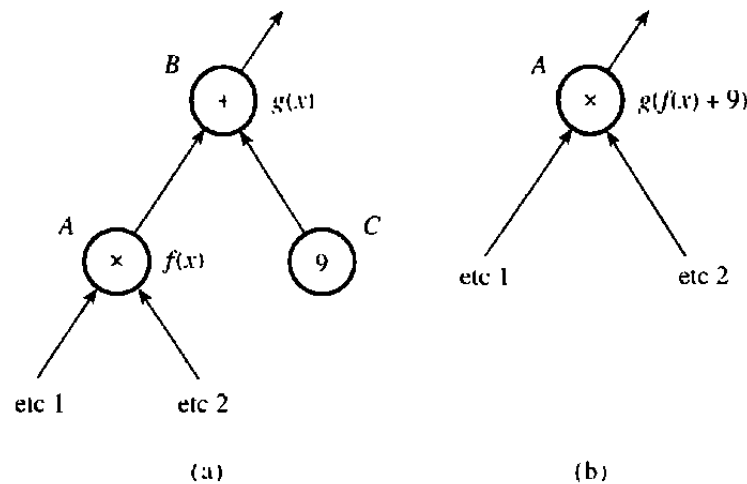


Figura 11.9. Un fragmento de un árbol de expresión

Incluso antes de recibir un valor del nodo A , sin embargo, el procesador del nodo B puede efectuar un trabajo útil, reconstruyendo el árbol de expresión. Dado que se conoce el valor del hijo derecho del nodo B , puede modificar la función almacenada en el nodo A y eliminar el nodo B y su hijo (el derecho, en este caso) del árbol. El resultado se muestra en la figura 11.9b. Si cuando calcule el valor de x el procesador del nodo A transmite directamente $g(f(x) + 9)$ al predecesor de B , soslayando a A y a su hijo derecho, entonces el resultado obtenido en la raíz del árbol no varía.

Nada hay de especial acerca del operador $+$ del nodo B , ni tampoco acerca del valor 9 que le dimos a su hijo derecho C . En general, si el nodo B contiene el operador $\circ$, en donde $\circ$ es cualquiera de entre $+$, $-$, $\times$ o $/$, y si el valor de su hijo derecho es cualquier constante k , entonces B puede sustituir la función de A por $g(f(x) \circ k)$ y eliminarse a sí mismo y a su hijo derecho del árbol de expresión. Si la constante k es el hijo izquierdo de B y el nodo A es el hijo derecho de B , entonces B debe de sustituir la función de A por $g(k \circ f(x))$. Esto es importante porque los operadores $-$ y $/$ no son conmutativos. Unos ajustes menores resuelven los casos en que A es una hoja (se toma como $f(x)$) la función constante que proporciona el valor del operando que hay en A) o bien cuando B es la raíz del árbol (el valor final del árbol es el valor que normalmente se elevaría hasta el nivel siguiente).

La operación descrita más arriba se llama *corte*. En el ejemplo, la hoja C y su predecesor B se han cortado del árbol. Es importante para lo que sigue ver que un corte se puede efectuar en tiempo constante. La manipulación de punteros del árbol se puede efectuar ciertamente en tiempo constante: sólo tres punteros están implicados en la operación. Resulta menos evidente que la función asociada a un nodo se pueda actualizar rápidamente. Por ejemplo, si en el ejemplo los nodos A y B se han creado ambos mediante una serie precedente de operaciones de corte, las funciones $f(x)$ y $g(x)$ podrían ya ser —o eso parece— bastante complejas, así que al sustituir f en g para obtener $g(f(x) \circ k)$ no sería trivial.

Afortunadamente, si los únicos operadores admitidos son $+$, $-$, $\times$ y $/$, las funciones que se pueden obtener de esta forma tienen todas ellas la forma $(ax + b)(cx + d)$. Para ver esto, observe primero que la función inicial de todos los nodos internos, a saber, x , se puede representar mediante $a = d = 1$ y $b = c = 0$. Si $f(x) = (a_1x + b_1) / (c_1x + d_1)$, y $g(x) = (a_2x + b_2) / (c_2x + d_2)$, entonces $g(f(x)) = (a_3x + b_3) / (c_3x + d_3)$, en donde es $a_3 = a_1a_2 + b_2c_1$ y existen otras expresiones igualmente sencillas para b_3 , c_3 y d_3 ; véase el problema 11.11. Por tanto, para representar cualquier función asociada con un nodo interno sólo hay que mantener las cuatro constantes correspondientes, a , b , c y d . Siempre que las constantes no sean demasiado grandes, la representación de cualquier composición funcional de la forma $g(f(x)^{\circ k})$ o $g(k^{\circ}f(x))$ se puede computar en tiempo constante siempre que sea necesario. Además, cuando el valor de x esté disponible, $f(x)$ también se puede calcular en tiempo constante. (La salvedad es necesaria porque en una expresión complicada las constantes pueden crecer exponencialmente. Sin embargo, aquí no consideraremos esta posibilidad.)

La última consideración antes de enunciar el algoritmo para evaluar expresiones es que en una operación de corte sólo están implicados tres nodos. En la Figura 11.9, estos nodos son A , B y C . Los punteros, valores y funciones asociadas se leen y se modifican para estos tres nodos y ninguno más. Por tanto es posible ejecutar una operación de corte en alguna otra parte del árbol, en paralelo con la operación que afecta a A , B y C , siempre y cuando ninguno de ellos esté implicado en la segunda operación. Además, sean A , B y C los nodos implicados en una operación de corte, y A' , B' y C' los nodos implicados en otra, siendo C y C' las dos hojas implicadas. Recuerde que las hojas del árbol de expresión están numeradas por orden en torno al árbol. Entonces es condición suficiente para que las operaciones no interfieran entre sí —esto es, para que los conjuntos $\{A, B, C\}$ y $\{A', B', C'\}$ sean disjuntos— el que C y C' sean hojas no consecutivas y que sean ambos hijos izquierdos o hijos derechos. El problema 11.12 pide al lector que demuestre esta afirmación.

El algoritmo paralelo completo para evaluar expresiones simples se puede enunciar ahora en la forma siguiente. Suponemos que se reserva un procesador para cada nodo interno del árbol de expresión:

función *evalp*(T : árbol de expresión)

{Evalúa un árbol de expresión con n hojas y $n-1$ nodos internos. Las hojas están numeradas inicialmente por orden siguiendo la base del árbol}

para cada nodo interno de T **hacer en paralelo**

 iniciar la función $f(x)$ con el valor x

repetir $\lceil \lg n \rceil$ veces

para cada nodo interno de T **hacer en paralelo**

 si el hijo izquierdo es una hoja impar, cortarlo

 si el hijo derecho es una hoja impar, cortarlo

 si alguno de los hijos es ahora una hoja, cambiar su número

devolver el valor de la hoja restante

En primer lugar, se cortan todas las hojas impares que sean hijos izquierdos. Por los comentarios anteriores, se puede hacer esto en paralelo sin que las operaciones de corte interfieran unas con otras. A continuación se cortan todas las hojas impares que sean hijos derechos. Una vez más, esto se puede hacer en paralelo. Dado que todas las hojas impares son o bien un hijo izquierdo o bien un hijo derecho, se habrán eliminado todas al llegar este momento, y solamente quedan las $\lfloor n/2 \rfloor$ hojas pares. Éstas se reenumeran dividiendo su número por dos, y estamos preparados para la próxima iteración. Dado que cada iteración elimina al menos la mitad de las hojas del árbol de expresión, al cabo de $\lceil \lg n \rceil$ iteraciones sólo queda una hoja. El valor de esta hoja es el valor de la expresión.

La figura 10.10 ilustra una iteración de este proceso cuando se le aplica al árbol de expresión de la figura 11.8. La primera mitad de la figura muestra el estado del árbol después de haber cortado las hojas izquierdas de número impar, y la segunda mitad muestra el estado después de haber recortado las hojas derechas de número impar. Se aprecia fácilmente que si ahora se dividen por dos los números de las hojas, el árbol quedará preparado para la próxima iteración. El lector puede verificar que la segunda iteración reduce el árbol a tres nodos (un nodo interno y dos hojas), mientras que la tercera lo reduce a un único nodo que contiene el valor de la expresión original, a saber, 24.

Dado que un corte se puede efectuar en tiempo constante, es fácil ver que el algoritmo descrito más arriba requiere un tiempo que está en $\Theta(\log n)$, empleando $\Theta(n)$ procesadores. El trabajo efectuado está en $\Theta(n \log n)$, así que el algoritmo no es óptimo. Tal como en los ejemplos anteriores, al cabo de una iteración sólo sigue siendo útil la mitad de los procesadores; después de dos iteraciones sólo actúa la cuarta parte, y así sucesivamente. A costa de una mayor complejidad, podemos

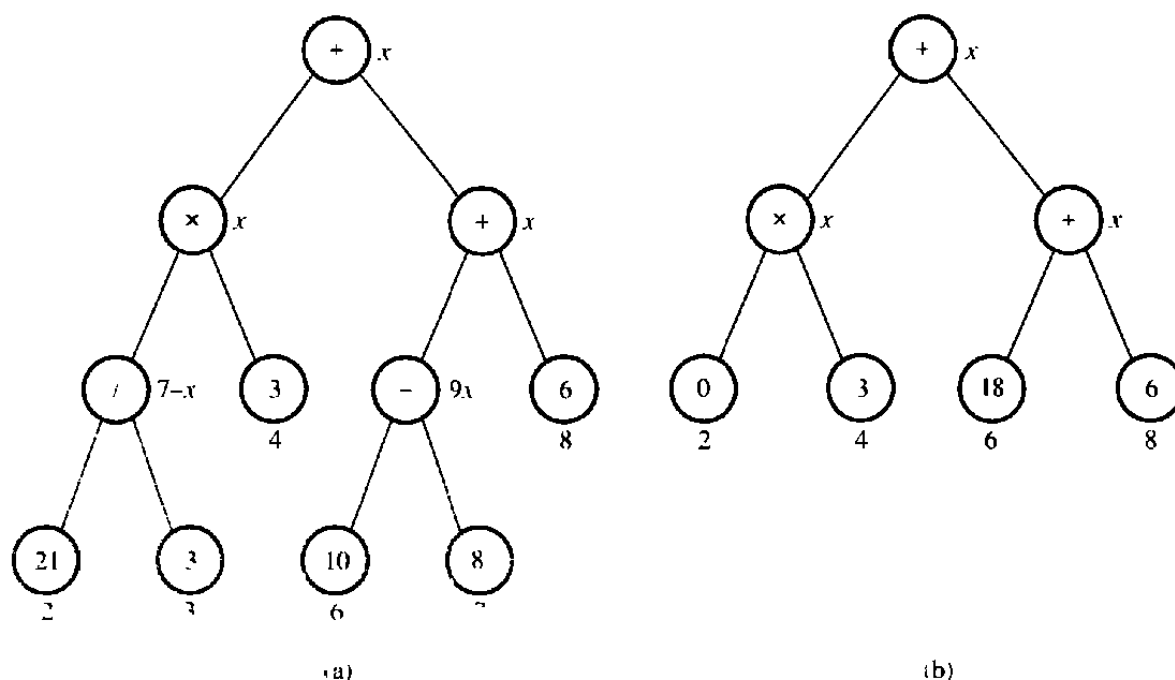


Figura 11.10. (a) Eliminados los hijos izquierdos (b) Eliminados los hijos derechos

aprovechar esto para reducir el número de procesadores necesario hasta $\Theta(n/\log n)$ sin que el tiempo requerido vaya más allá de $\Theta(\log n)$. El algoritmo mejorado funciona en $\Theta(n)$, y por tanto es óptimo.

La forma de la entrada necesaria para el algoritmo anterior (un árbol de expresión con las hojas numeradas de izquierda a derecha) puede parecer poco habitual. Aunque aquí omitiremos los detalles, observamos que si la expresión que hay que calcular no tiene la forma requerida, sino que está almacenada en forma de una cadena (esto es, un vector de caracteres) entonces el árbol de expresión numerado se puede obtener en un tiempo que está en $\Theta(\log n)$ empleando $\Theta(n/\log n)$ procesadores: son exactamente las mismos órdenes que para el algoritmo de evaluación. Por tanto, la conversión de la entrada a la forma requerida no constituye una pérdida de rendimiento.

11.6 REDES DE ORDENACIÓN EN PARALELO

Antes de utilizar el problema de la ordenación en paralelo empleando un *p-ram* CREW, haremos una momentánea digresión para examinar una interesante clase de redes que son capaces de ordenar sus entradas.

Comenzaremos por definir un *comparador*. Se trata de un circuito con dos entradas y dos salidas, que representaremos tal como en la figura 11.11. Por convención, suponemos que las entradas están en el lado izquierdo y las salidas en el derecho. Si como se muestra en la figura las dos entradas son x_1 y x_2 , y las dos salidas son y_1 e y_2 , entonces es $y_1 = \min(x_1, x_2)$ e $y_2 = \max(x_1, x_2)$. En otras palabras, la entrada mayor baja a la salida inferior, mientras que la entrada menor sube a la salida superior. En lo que sigue, será cómodo suponer que las entradas se intercambian cuando $x_1 \leq x_2$. De esta manera, es como si las entradas iguales se intercambiaran, aunque por supuesto esto no tiene ningún efecto visible. Claramente, un solo comparador es capaz de ordenar dos entradas. Supondremos que esta operación de comparación se puede efectuar en un tiempo constante. También suponemos que puede funcionar en paralelo cualquier cantidad de comparadores, siempre que sus entradas y salidas sean disjuntas. En la práctica, resulta fácil construir comparadores que satisfagan estos requisitos.

Más generalmente, deseamos diseñar redes que admitan n entradas. Si se aplica cualquier vector $(x_1, x_2, \dots, x_n)$ a las entradas de una de estas redes, entonces el vector de salida $(y_1, y_2, \dots, y_n)$ será una permutación de las entradas tal que $y_1 \leq y_2 \leq \dots \leq y_n$. Si denotamos mediante S_n una de estas redes, entonces S_1 no requiere ningún com-

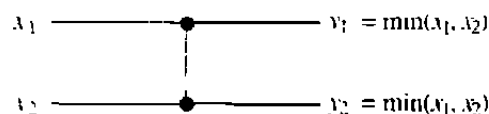


Figura 11.11. Un comparador

parador, mientras que S_2 , como acabamos de ver, es un único comparador. Una forma de construir redes cada vez más grandes consiste en diseñar S_{n+1} en términos de S_n , de tal manera que a partir de S_1 o de S_2 se pueden construir todas las redes que se deseen. Hay por los menos dos maneras evidentes de hacerlo, que se ilustran en la figura 11.12. Ambas redes poseen $n+1$ entradas y $n+1$ salidas. La red de la izquierda corresponde a ordenar por selección: el mayor elemento cae al fondo, y entonces utilizamos S_n para ordenar los n valores restantes. La red de la derecha corresponde a la ordenación por inserción: utilizamos S_n para ordenar las n primeras entradas, y entonces se inserta la $(n+1)$ -ésima entrada en el lugar adecuado. Curiosamente, cuando comparamos las redes obtenidas de esta manera, resultan ser la misma. La figura 11.13 ilustra la red S_5 que se obtiene tanto si se usa la inserción como empleando la selección.

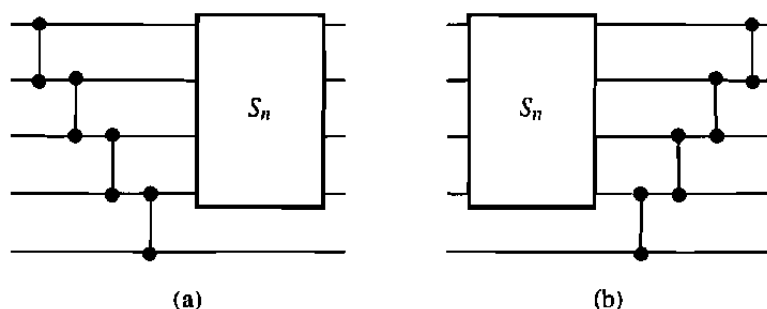


Figura 11.12. Redes de ordenación por selección e inserción

Existen dos medidas útiles de la calidad de nuestras redes. En primer lugar, podemos contar simplemente el número de comparadores necesarios para construir S_n . Esto se llama *tamaño* de la red. En el ejemplo, S_5 contiene 10 comparadores, y es evidente que en general S_n contendrá $\sum_{i=1}^{n-1} i = n(n-1)/2$ comparadores. La segunda medida de interés es el tiempo que necesita la red para ordenar sus entradas. Suponíamos que un comparador requiere un tiempo constante para funcionar, pero desde luego no puede activarse mientras sus salidas no estén preparadas. Definimos la *profundidad* de una red como el máximo número de comparadores a través de los cuales tiene que pasar una entrada antes de que llegue a una salida. La profundidad de la red que aparece en la figura 11.13 es 7. Los comparadores de la misma línea vertical pueden funcionar a la vez, mientras que las líneas sucesivas tienen que ejecutarse de izquierda a derecha. En este caso, la entrada 2 pasa por 7 comparadores. En general, resulta sencillo ver que una red S_n diseñada de esta manera tiene una profundidad $2n-3$ cuando $n \geq 2$. Si cada fase requiere un tiempo constante, el tiempo necesario para ordenar n elementos empleando este tipo de red de ordenación paralela es proporcional a la profundidad de la red, y por tanto está en $\Theta(n)$.

En las secciones siguientes veremos la forma de mejorar este diseño.

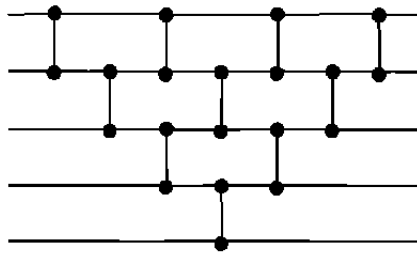


Figura 11.13. Una red para ordenar cinco entradas

11.6.1 El principio cero-uno

Para demostrar que nuestras redes de ordenación funcionan como deberían hacerlo, emplearemos la siguiente proposición, ligeramente sorprendente, y conocida con el nombre de *principio cero-uno*.

Lema 11.6.1. *Una red de ordenación con n entradas ordena correctamente cualquier conjunto de valores dado en sus entradas si y solo si ordena correctamente todos los 2^n vectores de entrada que constan únicamente de ceros y unos.*

Demostración La parte «sólo si» de la proposición es obvia. Para demostrar la parte “si”, sea $f: \mathbb{R} \rightarrow \mathbb{R}$ cualquier función no decreciente, de modo que $f(x) \leq f(y)$ siempre que $x \leq y$. Supongamos que la red de ordenación considerada ordena correctamente todos los 2^n vectores de entrada que constan solamente de ceros y unos, pero que existe algún vector de entradas $(x_1, x_2, \dots, x_n)$ que se ordena de forma incorrecta. Obsérvese que incluso una red de ordenación incorrecta produce una permutación de sus entradas. Sea $(y_1, y_2, \dots, y_n)$ el vector de salida (incorrecto) para este conjunto de entradas, y sea y_i cualquier elemento de este vector tal que $y_i > y_{i+q}$. Este elemento tiene que existir necesariamente, puesto que el vector está mal ordenado.

Consideremos ahora lo que sucedería si en lugar de $(x_1, x_2, \dots, x_n)$ aplicásemos a la red el vector de entrada $(f(x_1), f(x_2), \dots, f(x_n))$. Dado que f es no decreciente, $f(x_i) \leq f(x_j)$ siempre que sea $x_i \leq x_j$. De esta manera, los valores de $f(x)$ se propagan por la red de ordenación exactamente de igual manera que los valores de x ; del mismo modo que dos valores de x se intercambiaban, se intercambian ahora los dos valores correspondientes de $f(x)$ (esta es la razón por la cual exigíamos que los comparadores intercambiasen dos valores iguales.) El vector de salida de la red de ordenación será $(f(y_1), f(y_2), \dots, f(y_n))$, puesto que la red va a efectuar exactamente la misma permutación de sus entradas que en el caso anterior.

Por último, sea f la función definida en la forma siguiente: $f(x) = 0$ cuando $x < y_i$ y $f(x) = 1$ en caso contrario. Ahora el vector de entradas $(f(x_1), f(x_2), \dots, f(x_n))$ es un vector que consta únicamente de ceros y unos. En el vector de salida $f(y_i) = 1$ y $f(y_{i+1}) = 0$, porque $y_{i+1} < y_i$. El vector de salida, por tanto, estará incorrectamente ordenado, lo cual contradice nuestra suposición de que la red ordena correctamente todos los vectores de en-

trada que contengan únicamente unos y ceros. Se sigue que no puede existir ninguno de estos vectores $(x_1, x_2, \dots, x_n)$: la red ordena correctamente *cualquier* vector de entrada, y nuestra demostración del principio cero-uno está terminada.

11.6.2 Redes de fusión en paralelo

Necesitamos una herramienta más antes de volver a las redes de ordenación. Para todo entero positivo n , una *red de fusión* F_n es una red formada por comparadores, con dos grupos de n entradas y un solo grupo de $2n$ salidas. Siempre y cuando cada uno de los dos grupos de entradas esté ya ordenado, cada entrada aparecerá en una de las salidas, y las salidas también estarán ordenadas (compárese esto con la descripción de la fusión en la Sección 7.4.1). Por ejemplo, la figura 11.14 muestra una red F_4 , que ilustra la forma en que las entradas se transmiten a las salidas.

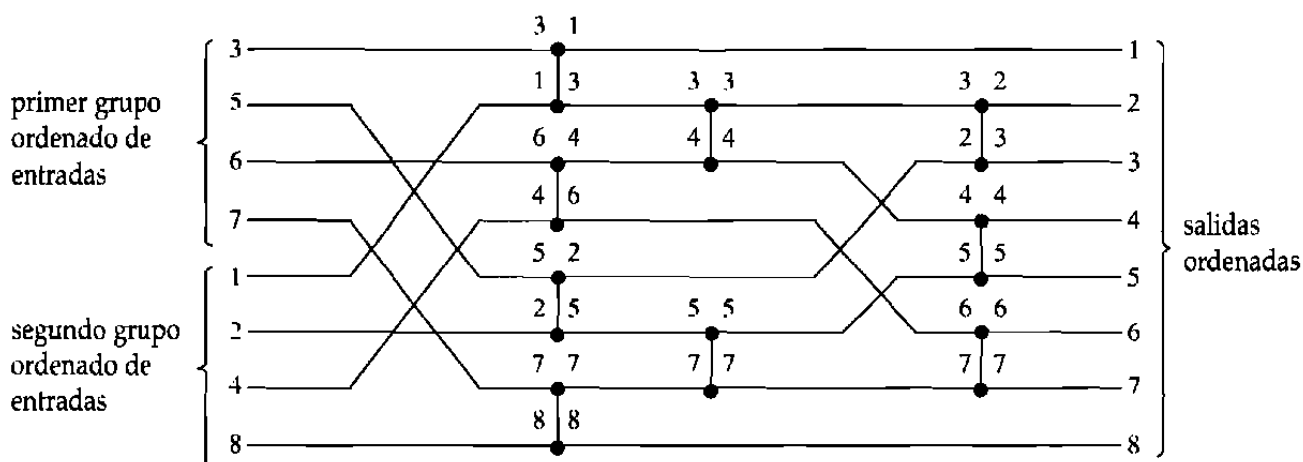


Figura 11.14. Una red de fusión

Por sencillez, supondremos a partir de ahora que n es una potencia de 2. Un solo comparador puede servir como F_1 . Partiendo de esta base creamos redes de fusión F_2 , F_4 y así sucesivamente, diseñando siempre a F_{2n} en términos de F_n . Este es otro ejemplo de la técnica de divide y vencerás que se describe en el Capítulo 7. La figura 11.15 muestra la forma de hacerlo. Supongamos que los dos grupos de entradas son $(w_1, w_2, \dots, w_n)$ y $(x_1, x_2, \dots, x_n)$, y que las salidas son $(y_1, y_2, \dots, y_{2n})$. Fusionamos las entradas impares $(w_1, w_3, \dots, w_{n-1})$ del primer grupo con las entradas impares $(x_1, x_3, \dots, x_{n-1})$ del segundo grupo empleando una red de fusión F_n , y fusionamos las entradas pares $(w_2, w_4, \dots, w_n)$ del primer grupo con las entradas pares $(x_2, x_4, \dots, x_n)$ del segundo grupo empleando otra red. Llamaremos a las salidas de estas dos fusiones $(v_1, v_2, \dots, v_{2n})$, numerando las salidas de la fusión impar antes que las salidas de la fusión par. Ahora se permutan las salidas v_i de tal manera que, desde arriba hasta abajo, se encuentren en el orden $(v_1, v_{n+1}, v_2, v_{n+2}, \dots, v_n, v_{2n})$. Esta permutación es lo que se denomina *barajado perfecto*: si cortamos un mazo de $2n$ cartas exactamente por la mitad, y las mezclamos de tal modo que vaya cayendo alternativamente una carta de cada mitad, el orden que se obtiene es éste. Por último, se ins-

talán comparadores entre lo que ahora son las salidas 2 y 3, 4 y 5, $2n-2$ y $2n-1$. La salida de la derecha es el vector ordenado que se deseaba, $(y_1, y_2, \dots, y_{2n})$.

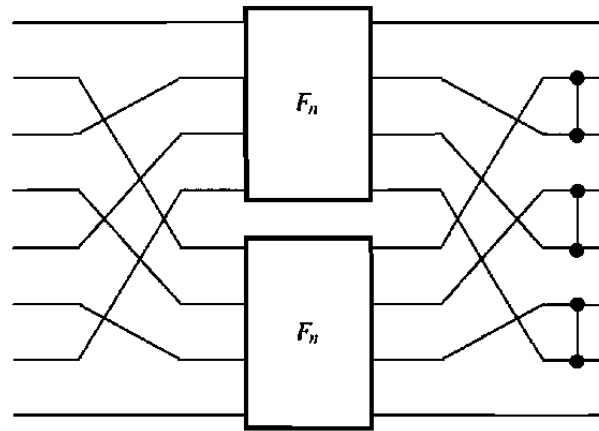


Figura 11.15. Diseño de F_{2n} en términos de F_n .

Un argumento exactamente análogo al dado anteriormente muestra que el principio cero-uno también es válido para redes de fusión. Por tanto, para demostrar que funciona la red propuesta, mostramos primero que funciona cuando las entradas w y x constan solamente de unos y ceros, y después invocamos el principio cero-uno para concluir que también funciona para entradas cualesquiera. El argumento es por inducción matemática. Como base para la inducción, es evidente que un único comparador, F_1 , funciona correctamente. Como paso de inducción, supongamos demostrado que todas las redes $F_1, F_2, \dots, F_n$ funcionan correctamente. Considérese lo que hace nuestro F_{2n} propuesto cuando el vector de entrada w consta de r ceros seguidos por $n-r$ unos, y x consta de s ceros seguidos por $n-s$ unos. (Recuerde que ambos grupos de entradas deben estar ya ordenados.) Dado que por la hipótesis de inducción las redes F_n funcionan correctamente, la salida $(v_1, v_2, \dots, v_n)$ de la red superior de fusión consta de $\lceil r/2 \rceil + \lceil s/2 \rceil$ ceros seguidos por el número adecuado de unos, mientras que la salida $(v_{n+1}, v_{n+2}, \dots, v_{2n})$ de la red de fusión inferior consta de $\lfloor r/2 \rfloor + \lfloor s/2 \rfloor$ ceros seguidos por unos. Si tanto r como s son pares, entonces después de barajar las salidas de las dos redes de fusión, las $2n$ líneas que van desde arriba hasta abajo contienen una vez más $r+s$ ceros seguidos por unos; tal como antes, la columna final de comparadores resulta innecesaria. Si tanto r como s son impares, sin embargo, entonces después de barajar las salidas de las dos redes de fusión, los valores de las líneas que van desde arriba hacia abajo son $r+s-1$ ceros, un uno, un cero, y después unos. Esta vez el comparador entre las líneas $r+s$ y $r+s+1$ es necesario para finalizar la ordenación. El F_{2n} propuesto ordena por tanto correctamente todas las entradas que consten únicamente de ceros y unos; por el principio cero-uno, ordenará correctamente entradas cualesquiera.

La figura 11.14 se obtuvo de esta manera, salvo que se ha retocado la red para eliminar algunos cruces de líneas redundantes.

Para calcular el tamaño $s(n)$ de la red F_n obtenida empleando esta construcción, utilizaremos la recurrencia

$$s(2n) = 2s(n) + n/2 - 1$$

que es inmediata a partir de la Figura 11.15. La condición inicial es $s(1) = 1$. Empleando los métodos de la Sección 4.7 se sigue fácilmente que $s(n) = 1 + n \lg n$. Resulta igualmente fácil mostrar que la profundidad de la red F_n es $1 + \lg n$.

11.6.3 Redes de ordenación mejoradas

Ahora podemos utilizar la técnica de divide y vencerás para diseñar redes de ordenación S_n que sean una mejora notable con respecto a las vistas al principio de la Sección 11.6. La figura 11.16 muestra como hacerlo: se utilizan dos redes S_n para ordenar por separado las n primeras entradas, y las n últimas, y después se utiliza un circuito de fusión F_n para completar la ordenación. No es necesario demostrar que la red S_{2n} funciona correctamente si las dos redes menores S_n funcionan también correctamente. El enfoque de divide y vencerás se detiene en S_2 , que consta de un solo comparador.

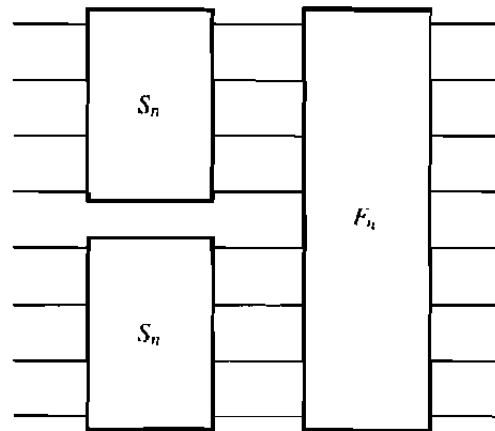


Figura 11.16. Una red de ordenación mejorada.

Resulta fácil mostrar que las redes de ordenación obtenidas de esta manera emplean un número de comparadores que está en $\Theta(n \log^2 n)$ y que el tiempo que necesitan para ordenar sus entradas está en $\Theta(\log^2 n)$. Por razones evidentes, las redes descritas más arriba se denominan redes par-impar de ordenación o fusión. Fueron descubiertas por Batcher en 1964. Estas redes de ordenación no son óptimas. Anticipándonos un poco, veremos en el capítulo siguiente que *cualquier* algoritmo que ordena n elementos mediante comparaciones entre ellos debe hacer al menos $\lceil \lg n! \rceil$ en el caso peor. Por tanto, toda red de ordenación para n elementos debe incluir al menos $\lceil \lg n! \rceil$ comparadores. Por ejemplo, toda red de ordenación para 16 elementos tiene que contener un mínimo de 45 comparadores. La red par-impar de ordenación S_{16} contiene 63 comparadores. Se conoce una red diferente que utiliza solamente 60 comparadores. ¡Parece que aún es mejorable! Tampoco en lo tocante a pro-

fundidad la red par-impar de ordenación es óptima: para 16 entradas, la red par-impar de ordenación tiene una profundidad 10, pero se conoce una red diferente que tiene profundidad 9. Se sabe que existe una clase de redes de ordenación cuya profundidad está en $\Theta(\log n)$ y cuyo tamaño está en $\Theta(n \log n)$, pero todavía no ha producido redes que resulten útiles en la práctica.

11.7 ORDENACIÓN EN PARALELO

Las secciones anteriores describen la forma de efectuar ordenaciones en paralelo empleando una red de comparadores. Si se simula cada comparador mediante un procesador, se obtiene un algoritmo que puede ejecutarse mediante un computador *p-ram* CREW. Este algoritmo puede ordenar n elementos en un tiempo que está en $\Theta(\log^2 n)$, empleando $\Theta(n \log^2 n)$ procesadores. (Esto se sigue directamente del resultado del problema 11.17). En esta sección se esboza un algoritmo de ordenación debido a Cole que se puede ejecutar en un *p-ram* CREW, y que ordena n elementos en un tiempo que está en $\Theta(\log n)$ empleando un número de procesadores que está en $\Theta(n)$. El trabajo efectuado está, por tanto, en $\Theta(n \log n)$. Tal como veremos en la Sección 12.2.1, este algoritmo paralelo es óptimo, al menos en lo tocante a la ordenación por comparación.

A lo largo de esta sección supondremos por sencillez que tenemos n elementos distintos, que deben clasificarse en orden ascendente, y donde n es una potencia de 2; en caso contrario, añadimos el número necesario de elementos mudos. En esencia, el algoritmo paralelo de Cole es una ordenación por fusión basada en árboles. Puede servir de ayuda compararlo con la ordenación por fusión secuencial ordinaria que se describe en la Sección 7.4.1. El algoritmo emplea un árbol binario completo con n hojas. Inicialmente, se dispone un elemento a ordenar en cada hoja. Entonces el cálculo progresa ascendiendo por el árbol, nivel por nivel, desde las hojas hasta la raíz. En cada nodo interno, los subconjuntos ordenados producidos por sus hijos se van fusionando. Supongamos que esta fusión se pueda efectuar en un tiempo que esté en $\Theta(M(n))$, y que todas las fusiones del mismo nivel del árbol se puedan efectuar en paralelo. Dado que hay $\lg n$ niveles de nodos internos en el árbol, todo el proceso completo de ordenación se puede efectuar en un tiempo que está en $\Theta(M(n) \log n)$.

Si no se dispone de información adicional acerca de dos sucesiones ordenadas cada una de las cuales contiene m elementos, se puede demostrar que con m procesadores el tiempo requerido para fusionar las secuencias está en $\Omega(\log \log m)$. Empleando esta aproximación, esperaríamos que una ordenación por fusión basada en árboles requiriera un tiempo que estuviese al menos en $\Omega(\log n \log \log n)$. La idea de Cole era proporcionar tan solo la información adicional necesaria para que fuera posible fusionar dos sucesiones ordenadas en un tiempo que estuviera en $O(1)$, de tal forma que el algoritmo global se ejecutase en un tiempo en $O(\log n)$.

La sección siguiente agrupa algunas definiciones necesarias, y después se esboza el algoritmo.

11.7.1 Preliminares

Hay que ordenar n elementos. A medida que progresa el algoritmo, se almacenan diferentes subconjuntos de estos n elementos por orden ascendente en vectores ordenados. En lo que sigue, dado que todos los vectores de interés están ordenados, nos limitamos a llamarlos vectores, y damos por supuesto que están ordenados los elementos que contienen. Utilizaremos letras minúsculas para denotar los elementos, y letras mayúsculas para denotar vectores.

Sean a, b y c tres elementos, con $a < c$. Decimos que b está entre a y c si $a \leq b < c$. También diremos que a y c rodean a b .

Sea L un vector de elementos. Si a es un elemento de L , entonces el rango de a en L se define de forma sencilla: el menor elemento de L tiene rango 1, el próximo en tamaño tiene el rango 2, y así sucesivamente. Suponemos tácitamente que todos los vectores L se aumentan con dos elementos invisibles, a saber, $-\infty$ que tiene rango 0 y $+\infty$, que tiene rango $|L| + 1$. Esto nos permite definir el rango de un elemento de un vector con respecto a otro (que a veces se denomina *rango cruzado*) en la forma siguiente. Sean L y J dos vectores, y sea b un elemento de J . Si a y c son los dos elementos consecutivos de L que rodean a b (y existen necesariamente, por existir los dos elementos invisibles de L), entonces se define el rango de b en L como igual al rango de a en L . Denotaremos mediante J/L el conjunto de rangos respecto a L de todos los elementos de J .

Una vez más, sean J y L dos vectores de elementos, y sean ahora a y b dos elementos consecutivos de L (incluyendo los elementos invisibles). Definimos el intervalo inducido por estos elementos en la forma $[a, b]$. Un elemento x pertenece a este intervalo si $a \leq x < b$, esto es, si a y b rodean a x . Diremos que L cubre a J si todo intervalo inducido por elementos consecutivos de L contiene como máximo tres elementos de J . (En esta ocasión, los elementos invisibles de J no se incluyen.) Por ejemplo, si L contiene los elementos 10, 20 y 30, mientras que J contiene a 1, 7, 22, 23, 26 y 35, entonces L cubre a J : el intervalo inducido por los elementos $-\infty$ y 10 de L contiene dos elementos de J , el intervalo inducido por 10 y 20 no contiene ninguno, y así sucesivamente. Sin embargo, si K contiene los elementos 11, 12, 14, 17, 22 y 41, entonces L no cubre a K porque el intervalo inducido por los elementos de 10 a 20 de L contiene más de tres elementos de K .

Utilizaremos el símbolo $\&$ para denotar la operación de fusión de dos vectores ordenados. Supongamos que L cubre a J , y que M cubre a K . En contra de lo que podría esperarse, no es necesariamente cierto que $L\&M$ cubra a $J\&K$; véase el problema 11.19. Finalmente, para todo vector L , $r(L)$ denota el vector que se obtiene tomando todos los cuartos elementos de L que ocupan posiciones múltiplos de cuatro. Si L contiene menos de cuatro elementos $r(L)$ está vacía.

11.7.2 La idea clave

Ahora daremos una descripción informal del método para fusionar dos vectores en un tiempo que está en $O(1)$. Sean J, K y L tres vectores ordenados, y supongamos que L cubre tanto a J como a K . La situación es la ilustrada en la figura 11.17.

Dado que L cubre a J , todo intervalo inducido por dos elementos consecutivos de L , incluyendo los inducidos por los elementos invisibles, contiene al menos tres elementos de J ; lo mismo se puede decir para K . Si conocemos los rangos J/L , K/L , L/J y L/K , es fácil determinar qué elementos se encuentran en cada intervalo.

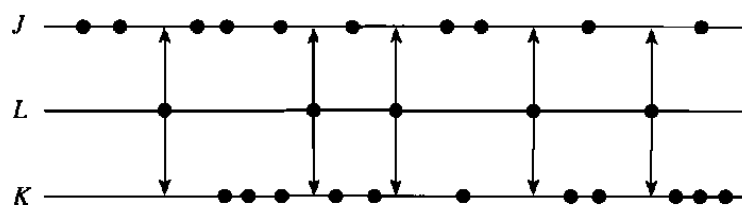


Figura 11.17. Fusión de J y K con ayuda de L

Los elementos de J situados en el primer intervalo inducido por L se pueden fusionar con los de K situados también en el primer intervalo en un tiempo que está en $O(1)$, porque hay como mucho seis de ellos en total. Lo mismo se puede decir de los elementos de J y de K situados en el segundo intervalo inducido por L , y así sucesivamente. Si disponemos de un número suficiente de procesadores, todas estas fusiones de un máximo de seis elementos se pueden efectuar en paralelo, así que será posible terminarlas todas en un tiempo que esté en $O(1)$. Para obtener $J \& K$, lo único que resta por hacer es concatenar los resultados de fusionar cada intervalo por separado. De esta manera, toda la fusión se puede completar en tiempo constante.

Llamaremos a esta operación *fusión con ayuda*. En el ejemplo anterior, decimos que los vectores J y K se fusionan con ayuda de L .

11.7.3 El algoritmo

Los n elementos que hay que ordenar se sitúan inicialmente en las hojas de un árbol binario completo. En un nodo interno típico v de este árbol, la tarea es calcular L_v , un vector que contiene todos los elementos del subárbol cuya raíz es v . en cada fase del cómputo en el nodo v , hay un vector A_v que contiene un subconjunto de los elementos de L_v ; A_v se recalcula en cada fase. Más exactamente, sea $A_v(t)$ el vector en cuestión al principio de la fase t , $t = 0, 1, \dots$, y sea $A_v(t+1)$ el vector creado durante esa fase. En general, $A_v(t+1)$ es de un tamaño doble al de $A_v(t)$, hasta que finalmente $A_v(t+1) = L_v$. Un nodo v en el cual $A_v(t) \neq L_v$ se denominará *activo*, mientras que un nodo v en el cual $A_v(t) = L_v$ se denomina *completo*. los vectores asociados con los hijos de v reciben los sufijos x e y : no importa a qué hijo se refiera cada sufijo. En cada fase están implicadas otros tres vectores: $B_v(t)$ es un vector que se transmite subiendo hasta el padre de v , y $B_x(t)$ y $B_y(t)$ son dos vectores que se reciben de los hijos de v .

En primera aproximación, el cálculo efectuado durante cada fase en todos los nodos internos v comprende las tres fases siguientes (véase la figura 11.18):

1. Calcular el vector $B_v(t) \leftarrow r(A_v(t))$ y enviarla árbol arriba al padre de v .

2. Leer las dos matrices $B_x(t)$ y $B_y(t)$ que los hijos de v acaban de enviar por el árbol.
3. Calcular $A_v(t+1) \leftarrow B_x(t) \& B_y(t)$. Esta operación de fusión se efectúa en tiempo constante, según se indicaba más arriba, con ayuda del vector $A_v(t)$, que, tal como veremos, cubre tanto a $B_x(t)$ como a $B_y(t)$.

Hay diferencias en las tres fases según el nodo v sea activo o completo. Sin embargo, para nuestros propósitos en este momento es innecesario dar más detalles; basta con tener en cuenta que

- ◇ en la fase 0, los nodos del nivel 1 leen los valores enviados por sus hijos, que son hojas, fusionan estos valores y pasan a ser completos,
- ◇ pasadas tres fases después de que un nodo llega a ser completo, su padre pasa a su vez a ser completo.

Dado que hay $\lg n$ niveles de nodos internos en el árbol, concluimos que el algoritmo posee $3 \lg n - 2$ fases.

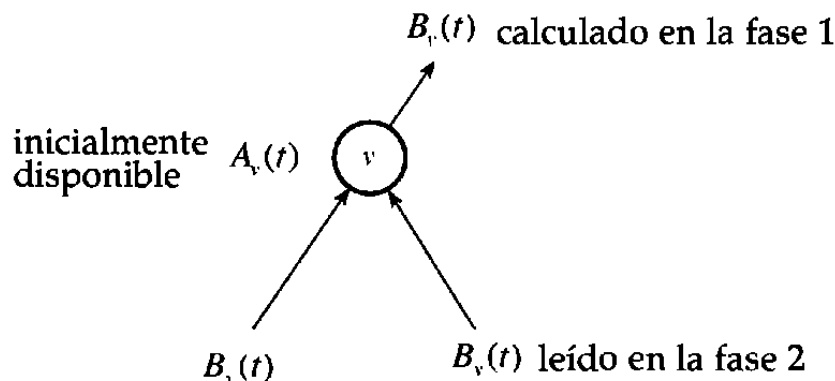


Figura 11.18. Fase t del nodo v

Queda por demostrar que las fusiones requeridas por la fase 3 de cada etapa se pueden efectuar en un tiempo que está en $O(1)$. Combinado con el resultado anterior, esto mostrará que el algoritmo completo se ejecuta en un tiempo que está en $O(\log n)$.

11.7.4 Un esbozo de los detalles

Para demostrar que en cualquier nodo interno v con hijos x e y en cualquier fase t se pueden fusionar los vectores $B_x(t)$ y $B_y(t)$ en un tiempo $O(1)$ con ayuda de $A_v(t)$, es preciso demostrar dos cosas: en primer lugar, que $A_v(t)$ cubre tanto a $B_x(t)$ como a $B_y(t)$; y en segundo lugar que los rangos $A_v(t)/B_x(t)$, $A_v(t)/B_y(t)$, $B_x(t)/A_v(t)$ y $B_y(t)/A_v(t)$ son conocidos, o bien se pueden determinar rápidamente.

Es bastante sencillo demostrar la propiedad de cobertura requerida. Hay un argumento largo pero no difícil que muestra que todos los rangos necesarios se pueden obtener en $O(1)$ siempre y cuando se asigne un procesador a cada elemento del vector $A(t)$ en todos los nodos activos, otro a cada elemento del vector $B(t)$ en todos los nodos activos, y por último otro a cada elemento del vector $A(t+1)$ en todos los nodos activos. Por tanto, todas las operaciones requeridas por una sola fase del algoritmo se pueden efectuar en tiempo en $O(1)$ siempre y cuando esté disponible un número de procesadores suficiente para asignar uno a cada uno de los elementos de los vectores $A(t)$, $B(t)$ y $A(t+1)$ en todos los nodos activos.

Para un nodo v , el tamaño del vector $A_v(t)$ es la cuarta parte del tamaño de los vectores $A_x(t)$ y $A_y(t)$ que tienen sus hijos, siempre y cuando los hijos todavía no sean completos. Por tanto el tamaño total de los vectores $A(t)$ en el nivel de v es de un octavo del tamaño de los vectores $A(t)$ en el nivel de sus hijos, siempre y cuando los hijos no sean completos. (Hay dos veces más hijos que padres). Aún cuando no hemos dado los detalles, esto también es cierto en la fase en que los hijos pasan a ser completos por primera vez. en la segunda fase en que los hijos son completos, el tamaño de $A_v(t)$ en v es la mitad del tamaño de $A_x(t)$ y $A_y(t)$ en sus hijos, así que el tamaño total de estos vectores en el nivel de v es la cuarta parte del tamaño que tienen en el nivel de los hijos; de forma similar, en la tercera fase en que los hijos son completos, el tamaño total de los vectores $A(t)$ en el nivel de v es la mitad del tamaño total de estos vectores en el nivel de los hijos.

Dado que nunca puede haber más de n elementos en todos los vectores $A(t)$ en un mismo nivel, el número total de elementos en todos los vectores $A(t)$ a lo largo de todo el árbol no puede superar el máximo de tres valores: $n + n/8 + n/16 + \dots = 8n/7$ cuando hay n elementos en el nivel cuyos nodos acaban de alcanzar el estado de completos, $n + n/4 + n/32 + \dots = 9n/7$ en la etapa siguiente, $n + n/2 + n/16 + \dots = 11n/7$ al cabo de otra etapa más. Por tanto, el tamaño total de todos los vectores $A(t)$ está acotado por $11n/7$. De manera similar, se puede mostrar que el tamaño total de todos los vectores $B(t)$, y por tanto el tamaño total de todos los vectores $A(t+1)$ en todos los nodos activos no es mayor que $8n/7$. El número total de elementos de todos los vectores $A(t)$, $B(t)$ y $A(t+1)$ en todos los nodos activos no será mayor, por tanto, que $27n/7$.

Concluimos finalmente que el algoritmo se puede ejecutar en un tiempo que está en $O(1)$ empleando un número de procesadores que está en $O(n)$. Tal como se ha descrito, el algoritmo no tiene conflictos de escritura, aunque pueden producirse lecturas simultáneas.

11.8 COMENTARIOS ACERCA DE LAS P-RAM EREW Y CRCW

A lo largo de este capítulo hemos supuesto que nuestros algoritmos se van a ejecutar en un *p-ram* CREW, esto es, en una máquina que permite a varios procesadores leer en una misma posición simultáneamente, pero que no permite la escri-

tura simultánea en una misma posición. Intuitivamente, parece probable que una máquina de estas características debería ser más potente que una p -ram EREW, y menos potente que una p -ram CRCW. Ahora daremos dos ejemplos sencillos que confirman esta intuición.

Para mostrar que una p -ram CREW puede tener menor rendimiento que una p -ram EREW, considere el problema siguiente. Un árbol binario contiene n nodos, y todo nodo i salvo la raíz está enlazado con su antecesor mediante un puntero $p[i]$. En la raíz, el puntero p tiene el valor especial **nil**. Todo nodo i posee además un segundo puntero $r[i]$. Deseamos fijar el valor de r en todos los nodos de tal manera que apunte a la raíz del árbol. Considere el siguiente algoritmo paralelo para hacerlo.

procedimiento *buscar-raíz*(T)
 { T es un árbol binario con n nodos}
para todo par ordenado (i, j) **hacer en paralelo**
 si $p[j] = \text{nil}$ **entonces** $r[i] \leftarrow j$

Aquí tanto i como j varían entre 1 y n , así que necesitamos n^2 procesadores para ejecutar la instrucción **si** en paralelo para todo par de nodos. Suponiendo que dispongamos de ellos, la instrucción se puede ejecutar en paralelo para todos los pares de nodos en un p -ram CREW. Cuando se evalúa la condición, n procesadores leen el valor de $p[j]$ para todo j ; sin embargo, se admiten las lecturas concurrentes, así que esto no es problema. En lo tocante a la instrucción de asignación, sólo un procesador para cada valor de i llega a la conclusión de que la condición es verdadera, así que para cada valor de i sólo un procesador le asigna un valor a $r[i]$. Por tanto, no hay conflictos de escritura. Las operaciones implicadas son elementales, así que el algoritmo puede ejecutarse en un tiempo que está en $O(1)$ empleando n^2 procesadores CREW.

Por otra parte, un sencillo argumento muestra que ningún algoritmo para un procesador EREW puede producir el resultado apetecido en un tiempo que esté en $O(1)$, independientemente del número de procesadores disponibles. Para cada paso en una máquina de estas características, sólo un elemento de información almacenado en memoria, como máximo, puede ser leído y copiado por un único procesador. Por tanto, el número de posiciones de memoria que puede contener este elemento de información tan sólo puede doblarse, como máximo, en cada paso. En el problema anterior, la identidad de la raíz es conocida inicialmente por un nodo como máximo. Para copiar esta información y almacenarla en los n nodos, se necesita por tanto un mínimo de $\lceil \lg n \rceil$ pasos. Por tanto, cualquier algoritmo para este problema que utilice procesadores EREW requiere un tiempo que está en $\Omega(\lg n)$.

Un segundo problema sencillo servirá para mostrar que una máquina CRCW puede tener mayor rendimiento que una máquina CREW. El problema consiste en hallar el máximo de n números que están almacenados en un vector $A[1..n]$. Considere el algoritmo siguiente.

```

función hallar max( $A[1..n]$ )
  matriz  $M[1..n]$  de Boolean
  para  $1 \leq i \leq n$  hacer en paralelo  $M[i] \leftarrow \text{verdadero}$ 
  para todos los pares ordenados  $(i,j)$  hacer en paralelo
    si  $A[i] < A[j]$  entonces  $M[i] \leftarrow \text{falso}$ 
  para  $1 \leq i \leq n$  hacer en paralelo
    si  $M[i]$  entonces  $\text{max} \leftarrow A[i]$ 
  devolver max

```

Tal como antes, i y j varían entre 1 y n , así que necesitamos n^2 procesadores para ejecutar la primera instrucción **si** en paralelo para todas las parejas de elementos del vector A . Suponiendo que estén disponibles, la instrucción se puede ejecutar en un tiempo que está en $O(1)$. Sin embargo, aquí pueden intentar escribir simultáneamente en $M[i]$ hasta $n-1$ procesadores. La máquina, por tanto, debe admitir las operaciones concurrentes de escritura. Obsérvese que si están implicados varios procesadores, todos ellos intentarán escribir el mismo valor: a saber, *falso*. Esta restricción adicional suele imponérsele al modelo CRCW. Tras la ejecución de la primera instrucción **si**, $M[i]$ es *verdadero* si y sólo si $A[i]$ es igual al mayor valor de A . (Puede haber varios de estos elementos.) Finalmente, la segunda instrucción **si** escribe este valor máximo en *max*. Una vez más, es posible que varios procesadores intenten escribir simultáneamente en *max*, pero si lo hacen, entonces todos ellos intentarán escribir el mismo valor. También esta instrucción se puede ejecutar en un tiempo que está en $O(1)$. Por tanto, todo el algoritmo se puede ejecutar en un tiempo que está en $O(1)$ en un *p-ram* CRCW que utilice n^2 procesadores.

Aun cuando el argumento no resulta sencillo en este caso, se puede demostrar que cualquier algoritmo (como los que se describían anteriormente en este capítulo) para hallar el máximo de n elementos utilizando un *p-ram* CREW debe emplear un tiempo que esté en $\Omega(\log n)$, independientemente del número de procesadores que estén disponibles. Por tanto, un *p-ram* CRCW es más potente que un *p-ram* CREW.

11.9 COMPUTACIÓN DISTRIBUIDA

En la Sección 11.1 definíamos un modelo de computación paralela llamado «modelo de instrucción única y múltiples flujo de datos», en el cual los procesadores están más o menos sincronizados. Supongamos que relajamos esta instrucción, y permitimos que cada procesador opere sobre sus propios datos, a su propia velocidad, y empleando su propio programa. Cuando tenga que informar acerca de algo interesante, enviará un mensaje a sus colegas, y quizá reciba mensajes de ellos en algunas ocasiones. Éste es el modelo de *múltiples instrucciones y múltiples flujo de datos*. Los dos modelos suelen denominarse SIMD y MIMD, respectivamente.

Con el modelo SIMD, aunque no sea una necesidad lógica, suele resultar conveniente mantener muy próximos a los procesadores implicados en la ejecución de un algoritmo paralelo, quizá incluso en un mismo dispositivo físico. En caso con-

trario, el coste adicional asociado a su sincronización puede resultar prohibitivo. Una vez que se relajan las restricciones, sin embargo, ya no hay razón para que todos los procesadores implicados en la ejecución de un algoritmo paralelo tengan que hallarse todos ellos en una misma ubicación. Si los mensajes se intercambian con frecuencia relativamente escasa, los procesadores pueden estar en cualquier parte del mundo, y los mensajes se pueden enviar por correo electrónico. La ejecución de un algoritmo de este tipo es un ejemplo de lo que se llama *computación distribuida*. Dos ejemplos sorprendentes de esta técnica tienen que ver respectivamente con la factorización de enteros muy grandes y al problema del viajante.

Para el primer ejemplo, considérese el algoritmo de Las Vegas para factorizar enteros muy grandes, que se describe en la Sección 10.7.4. El algoritmo depende esencialmente de hallar un número suficiente de enteros con una propiedad especial (sus cuadrados módulo n tienen que ser k -suaves (véase la descripción del algoritmo en la Sección 10.7.4), donde los candidatos se seleccionan aleatoriamente. Si están disponibles varios procesadores, es evidentemente posible hacer que uno de ellos recoja los enteros necesarios, mientras que los otros van cribando los posibles candidatos. Cada procesador puede trabajar independientemente de los demás, salvo quizá algunos acuerdos para evitar una innecesaria duplicación de esfuerzos. Cuando se encuentra un entero apropiado, el procesador en cuestión puede enviar un mensaje con la nueva información al encargado de la colección, hasta que finalmente se haya encontrado un número suficiente de enteros. Si el hallazgo de un entero adecuado es un suceso relativamente infrecuente, entonces el correo electrónico tiene una velocidad perfectamente suficiente para transmitir los mensajes.

Empleando este tipo de técnicas, Lenstra y Manasse diseñaron un experimento que consistió en el reclutamiento de voluntarios con acceso a Internet, en suministrarles los programas necesarios y en recoger los resultados a medida que estos iban apareciendo. El experimento comenzó en el verano de 1987, empleando el método de curva elíptica para la factorización, que no describimos en este libro. En 1988 cambiaron a una variante del algoritmo de criba cuadrática que se esboza en la Sección 10.7.4. Sus programas estaban diseñados para ejecutarse en una estación de trabajo durante aquellos períodos en que normalmente estaría desocupada —por la noche, o durante los fines de semana, por ejemplo— de tal manera que la potencia de cálculo empleada era prácticamente gratuita. Estiman que tuvieron acceso al equivalente de unos 1.000 millones de instrucciones por segundo de potencia de cálculo sostenida, lo cual les permitía factorizar enteros de 100 cifras en alrededor de una semana. Esperaban que el tiempo de cálculo se duplicase aproximadamente por cada tres cifras que se añadieran al número que había que factorizar. Véase la Sección 10.7.4 para un ejemplo reciente de un éxito todavía más impresionante para un número de 129 dígitos.

En el caso del problema del viajante, todos los programas de computadora eficientes se basan en un método que consiste en hallar planos de corte adecuados. (No se preocupe si no sabe lo que son.) Una vez más, se pueden utilizar varios procesadores para buscar planos de corte independientemente. En 1993, un equipo de